The LNM Institute of Information Technology, Jaipur

Software Development Laboratory

Course Code: CS314

Academic Audit Portal

Lab Assignments 1, 2, 3, 4,5,6 and 7 — Consolidated Solution

Team ID	T094
Team Members	Ayush Gupta, Aayana Jain, Daksh Jain, Dhruv Shankar Dandotiya
Roll Numbers	24ucs261,24ucs262,24ucs263,24ucs264
Course	CS314 — Software Development Laboratory

The LNM Institute of Information Technology, Jaipur Software Development Laboratory
Course Code: CS314
Lab Assignment 1: Solution – T094 Academic Audit Portal

1. Project Identification

Item	Details
Project Name	Academic Audit Portal (AAP)
Project Type	Web-based Academic Audit Management System
Team ID	T094
Team Members	Ayush Gupta, Aayana Jain, Daksh Jain, Dhruv Shankar Dandotiya
Roll Numbers	24ucs261,24ucs262,24ucs263,24ucs264
Development Model	Agile + Incremental Development
Primary Users	Auditors / Audit Committee, Department & Program Coordinators, Course Instructors, Administrators, Quality Assurance Cell
Development Team	Student software development team
Course	Software Development Laboratory (CS314)
Project Objective	To develop a centralized platform for managing academic audits of departments, programs and courses, covering audit criteria, evidence submission, observations, corrective actions and closure tracking
Expected Outcome	A working, tested and documented software system developed incrementally

2. Project Background

Current Problem

Academic audits of departments, programs and courses (for internal quality checks or accreditation frameworks such as NAAC/NBA) are typically conducted using manual and disconnected mechanisms such as printed checklists, emails, spreadsheets, physical file submissions and in-person meetings. This makes it difficult to track which audit criteria have been addressed, to collect and verify evidence, to record observations consistently, to assign and follow up on corrective actions, and to confirm final closure of audit findings.

Proposed Solution

The Academic Audit Portal (AAP) will provide a centralized platform for managing the complete academic audit lifecycle — from defining audit criteria and audit cycles, to evidence submission by departments/programs/courses, recording of observations by auditors, creation and tracking of corrective action plans, and final closure verification with reporting.

3. Project Vision

The Academic Audit Portal aims to provide a centralized, transparent and structured platform that enables institutions to plan, conduct, track and close academic audits of departments, programs and courses efficiently and with a verifiable audit trail.

4. Problem Statement

Educational institutions periodically audit departments, programs, and courses against defined quality criteria. Existing processes often rely on paper-based records and fragmented digital tools, making it difficult to manage criteria, collect evidence, record observations, track corrective actions,

and verify closure. The Academic Audit Portal provides an integrated platform to streamline these activities.

5. Project Objectives

The initial objectives should be:

1. To provide a centralized platform for defining audit cycles and audit criteria for departments, programs and courses.
2. To allow department/program coordinators and course instructors to submit evidence against each audit criterion.
1. To enable auditors to record observations and findings during an audit against specific criteria.
2. Classification of observations (major non-conformity, minor non-conformity, observation, best practice)
3. To support creation and assignment of corrective action plans in response to observations.
4. To track the status and due dates of corrective actions until closure.
5. To maintain role-based access control for auditors, coordinators, course instructors, administrators and the Quality Assurance Cell.
6. To provide notifications/reminders for pending evidence submissions, observations and corrective action deadlines.
7. To generate audit reports and dashboards summarizing audit and closure status.
10. To develop the system incrementally using sound software engineering practices.

6. Stakeholders

Stakeholder	Interest / Role
Department / Program Coordinators	Submit evidence against audit criteria and respond to observations
Course Instructors	Provide course-level evidence and implement corrective actions for their courses
Auditors / Audit Committee	Define audit criteria, conduct audits, record observations and verify closure
Administrator	Manage users, roles, audit cycles and overall system configuration
Quality Assurance Cell / Institute	Monitor audit status across departments and obtain consolidated reports
System Development Team	Design, develop, test and maintain the Academic Audit Portal

7. Initial Scope

The Academic Audit Portal will initially cover:

- User registration/login
- Role-based access (Auditor, Coordinator, Course Instructor, Administrator, Quality Assurance Cell)
- Audit cycle and audit criteria/checklist definition
- Assignment of departments/programs/courses to an audit cycle
- Evidence submission (document/file upload) against each criterion
- Recording of observations/findings by auditors
- Classification of observations (major non-conformity, minor non-conformity, observation, best practice)

- Creation and assignment of corrective action plans
- Tracking of corrective action status through to closure
- Notifications for pending submissions, observations and deadlines
- Basic audit-status reports/dashboard

Initially Out of Scope

To keep the project manageable, we should initially exclude things such as:

- Direct integration with external accreditation portals (e.g., NAAC/NBA submission systems)
- Automated plagiarism or document-authenticity verification
- Payment processing
- Complete ERP functionality
- Advanced analytics or AI-based risk prediction
- Dedicated mobile application
- Multi-institution/federated audit management

8. Initial Assumptions

- Users will have access to the Internet.
- Users will have valid institutional accounts or credentials.
- Auditors are authorized and trained members of the audit committee.
- Audit criteria/checklists are defined before an audit cycle begins.
- Evidence documents will be submitted in standard digital formats (PDF, DOC, images).
- The system will initially be developed as a web application.
- The system will use a centralized database.

9. Initial Constraints

- Limited development time
- Student development team
- Limited access to real institutional audit data
- Limited infrastructure during development
- Requirement changes during development
- Need to protect confidential audit data and evidence
- Limited testing with real auditors/departmental staff

10. Initial Success Criteria

The Academic Audit Portal will be considered successful if the team can demonstrate that:

- different user roles can access appropriate functionality;
- an auditor can define audit criteria for a department/program/course;
- a coordinator can submit evidence against assigned criteria;
- an auditor can record and classify observations during an audit;
- a corrective action plan can be created and assigned to a responsible stakeholder;
- the closure of a corrective action can be tracked and marked complete;
- an audit-status report/dashboard can be generated;
- important system functions are tested;

- defects are documented and resolved;
- the project is maintained in a controlled Git/GitHub repository;
- the final system is accompanied by appropriate software engineering documentation.

AI Usage Policy - Student Declaration

We declare that:

- We have disclosed the use of AI tools, if any, during preparation of this assignment.
- AI-generated content, if used, has been reviewed, verified, edited and adapted by the team.
- We understand the project sufficiently to explain and defend all submitted content during a viva/discussion.
- We have not submitted AI-generated content blindly or without verification.
- We have not used AI to misrepresent work performed by the team.
- The final document reflects the team's own project understanding and decisions.

AI Tool(s) Used	Claude (Anthropic)
Purpose of AI Use	Assisted in adding relevant features and functionalities for the **Academic Audit Portal**.
Team ID	T094
Team Member Names	Ayush Gupta, Aayana Jain, Daksh Jain, Dhruv Shankar Dandotiya
Date	31/08/2026

Team Declaration: *We confirm that the information provided above is accurate and that we accept responsibility for the submitted work.*

Team Representative Signature/Confirmation (write full names in UPPERCASE only):

AYUSH GUPTA

AAYANA JAIN

DAKSH JAIN

DHRUV SHANKAR DANDOTIYA

The LNM Institute of Information Technology, Jaipur
Software Development Laboratory
Course Code: CS314

Lab Assignment 2: Solution

11. User Classes

11.1 Faculty

Faculty are primary sources of audit document creation. Teachers will place audit documents in a clean and clear location.
They shall be able to:

- Authenticate;
- Give audit documents;
- Ask for items needed to educate;
- Check auditing team's needs;
- Check if audit team asks questions, concerns, if audit documents have been submitted;
- Check infrastructure, example: PCs in labs, smartboards, software, speakers, etc;
- Check if syllabus is completed or not;
- Check attendance of students;
- Resubmit audit documents;
- Ask students if lab or classroom resources are used;
- Check if audit documents have been given;

11.2 Audit team

Audit team will make audit questions.
They shall be able to:

- Check audit documents;
- Mark compliance/non compliance;
- Add observations;
- Recommend corrective actions;
- Review if corrective actions are taken or not;
- Closing observations;

11.3 HOD

Monitoring and validating reports of department activities. Checking attendance.

- View audit schedule
- View audit criteria
- Coordinate evidence submission
- Upload evidence
- Reply to observations
- Assign responsibility for taking corrective actions to faculty
- Submit corrective action evidence
- Track events

11.3 Administrative staff

Make different activity based events. The administrator shall:

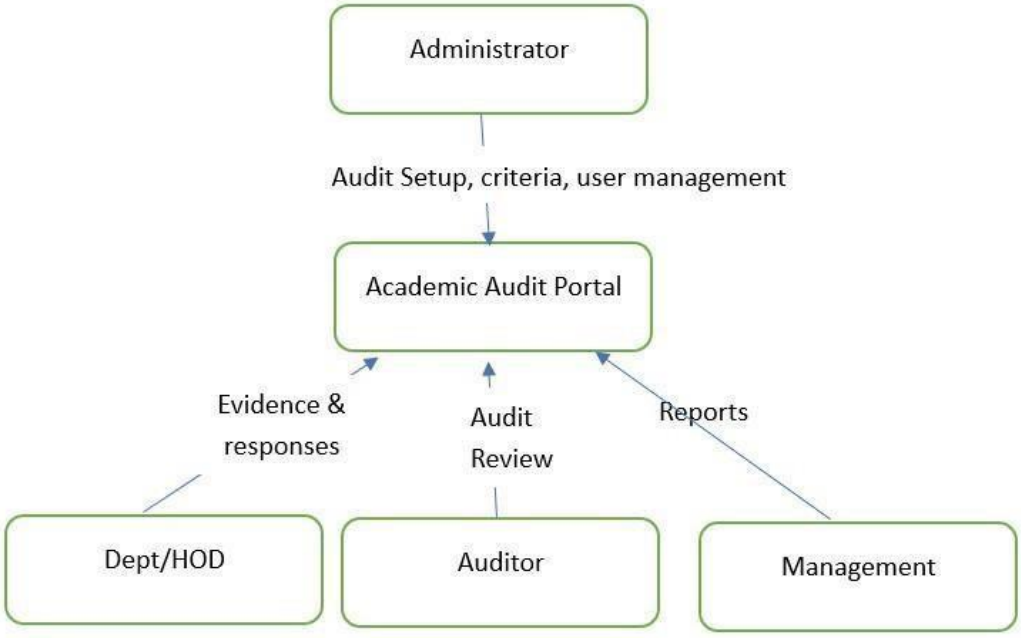
- Manage users/user-ids;
- Assign roles and access rights;
- Manage system level configuration;

- Monitor system activities;
- View administrative reports;

11.4 Management

View general schema. Management users shall have access to aggregate or consolidated dashboards.

12. System Context



13. Functional Requirements

13.1 Authentication and Account Management

FR-001 — User Login

The system shall allow a registered user to authenticate using valid credentials.

FR-002 — Authentication Failure

The system shall notify the user when authentication fails without unnecessarily revealing sensitive authentication information.

FR-003 — Logout

The system shall allow authenticated users to do logout.

FR-004 — Role Assignment
The system shall allow authorized administrators to assign roles to users.

FR-005 — Role-Based Access
The system shall restrict system functions according to the user's assigned role.

13.2 Department, Program and Course Management

FR-006: Department Management
Admin shall be able to:
Add departments
Edit department information
View departments
Activate/deactivate departments

FR-007: Program Management
Admin shall be able to add/delete programs of departments.

FR-008: Course Management
Admin or authorized department personnel shall be able to maintain courses associated withaprogram.

13.3 Audit Management

FR-009: Create Audit
Admin shall be able to create an academic audit. The audit shall contain:
Audit ID
Audit title
Audit type
Academic year
Department/program/course being audited
Audit start date
Audit end date
Audit coordinator
Audit team
Description
Status

FR-010: Audit Target
The system shall allow the Admin to specify what is being audited:
Department
Program
Course

FR-011: Assign Audit Team

Admin shall be able to assign one or more auditors to an audit. The system shall maintain:
Auditor name
Role
Audit assignment
Assignment date

FR-012: Audit Status
The system shall track the audit through different stages of Draft, EvidenceCollection, Corrective Action, Closure Review, etc. Only authorized users shall be able to change the auditstatus.

13.4 Audit Criteria Management
FR-013: Create Audit Criteria
Admin shall be able to define criteria against which an academic unit will be audited.
Each criterion may contain:
Criterion ID
Criterion name
Description
Category
Requirement
Applicable target
Evidence requirement
Priority

FR-014: Categorize Criteria
Criteria shall be grouped into categories.
Academic Planning
Teaching & Learning
Assessment
Student Performance
Attendance
Documentation
Infrastructure
Quality Improvement

FR-015: Assign Criteria to Audit
Admin shall be able to select the applicable criteria for a particular audit.

FR-016: Resubmit Audits:
Admin shall be able to resubmit audit/audit criteria in the portal, get past history.

13.5 Evidence Management

FR-017: Evidence Submission
The Department/HOD or Faculty shall be able to submit evidence against an audit criterion.
Evidence may include:

PDF
DOC/DOCX
XLS/XLSX
Images
Other permitted document formats

FR-018: Evidence Description
When submitting evidence, the user shall be able to provide:
Evidence title
Description
Document/file
Remarks

FR-019: Evidence Review
The Auditor shall be able to:
View evidence
Download evidence
Review evidence
Accept evidence
Reject evidence
Request additional evidence

FR-020: Evidence Versioning
If evidence is rejected or needs modification, the department shall be able to submit arevised version. The system should preserve the history rather than simply deleting the previous file.

FR-021: Evidence Status
Each evidence submission shall have a status such as:
Pending Review
Accepted
Rejected
Revision Required

13.6. Audit Evaluation
Once evidence is submitted, the auditor evaluates the criterion.

FR-022: Criterion Evaluation
The auditor shall be able to record an evaluation for each criterion. Outcomes can be:
Compliant
Partially Compliant
Non-Compliant
Not Applicable

FR-023: Auditor Comments
The auditor shall be able to add comments or remarks while evaluating a criterion.

13.7 Observation Management

FR-024: Create Observation
The auditor shall be able to create an observation based on the evidence review. An observation shall contain:
Observation ID
Audit ID
Criterion
Description
Severity
Auditor
Date
Status

FR-025: Observation Severity
The system may classify observations as:
Low
Medium
High

FR-026: Observation Status
The system shall track observations through their lifecycle:

FR-027: Observation Comments
Authorized users shall be able to add comments or responses to an observation. The system should maintain a history of comments.

FR-28: Faculty notified if audit's resubmit is done
If an audit is resubmitted by admin, faculty shall be notified using pop ups.

13.8. Corrective Action Management
This is the mechanism through which the department fixes audit findings.

FR-029: Create Corrective Action
For an observation requiring action, the auditor or authorized administrator shall create a corrective action. It should contain:
Action ID
Observation ID
Action description
Responsible person
Department
Due date
Priority
Status

FR-030: Assign Responsible Person
The corrective action shall be assigned to an appropriate person, such as:
HOD
Faculty
Course Coordinator
Department Coordinator

FR-031: Corrective Action Submission
The responsible user shall be able to:
Enter the action taken
Upload supporting evidence
Add comments
Submit the corrective action

FR-032: Corrective Action Review
The auditor shall review the submitted corrective action. The auditor can:
Approve
Reject
Request modification
Add comments

13.9 Closure Tracking

FR-033: Track Action Status
The system shall display the current status of every corrective action.

FR-034: Close Observation
An observation shall be marked Closed only after the auditor verifies that the correctiveaction has been satisfactorily completed.

FR-035: Reopen Observation
If the corrective action is inadequate, the auditor shall be able to reject it or reopen the observation.

FR-036: Overdue Actions
The system shall automatically identify corrective actions whose due dates have passed without closure.

13.10. Dashboard
Each role should have an appropriate dashboard.

FR-037: Admin Dashboard
Display:
Total audits
Active audits
Completed audits

Open observations
Overdue corrective actions
Closed corrective actions
Department-wise status

FR-038: Auditor Dashboard
Display:
Assigned audits
Pending evidence reviews
Open observations
Corrective actions awaiting verification
Upcoming audit deadlines

FR-039: HOD Dashboard
Display:
Department's active audits
Evidence pending submission
Observations
Corrective actions
Overdue actions
Closure percentage

FR-040: Faculty Dashboard
Display:
Assigned courses
Evidence requests
Assigned corrective actions
Pending submissions
Action deadlines

FR-41: Management Dashboard
Display high-level information regarding**:**
Total Audits
Completed
In Progress
Open Observations
Overdue Actions
Closure Rate

13.11 Reports

FR-042: Audit Report
The system shall generate an audit report containing:
Audit details
Audit team
Criteria

Evidence submitted
Auditor evaluations
Observations
Corrective actions
Closure status

FR-043: Observation Report
The system shall generate a report showing:
Total observations
Open observations
Closed observations
Overdue observations
Severity-wise distribution

FR-044: Corrective Action Report
The system shall show:
Total actions
Completed actions
Pending actions
Overdue actions
Closure rate

FR-045: Export Reports
The system may allow authorized users to export reports as:
PDF
Excel/CSV

13.12. Audit History and Tracking
FR-046: Audit History
The system shall maintain the history of an audit.

FR-047: Activity Log
The system shall maintain an activity log showing:
User
Action performed
Date/time
Related audit
Previous status
New status

Business rules

ID	Business Rule
BR-001	Only authorized users can create audits.

BR-002	An audit must be viewable.
BR-003	Latest versions of audits are to be used.
BR-004	Addition of comments must be possible.
BR-005	Notifications must be sent if the website is accessed in cases of resubmissions of audits.
BR-006	Questions and comments can be raised by faculty.
BR-007	Faculty/admin should be notified if an audit is submitted.
BR-008	Faculty/admin should be able to check the status of the audit.
BR-009	Faculty/admin should be able to check reports of audit team's.
BR-010	Faculty cannot edit the audit team's criteria.
BR-011	Admin can submit notifications to faculty.
BR-012	Faculty can submit notifications to admins.

15. Non-Functional Requirements

15.1 Performance

NFR-001: The system should provide acceptable response times for normal user operations.

NFR-002: Notifications should be sent and received instantly or with a delay, if the admin wants it, only if a portal is used.

NFR-003: System should support concurrent users.

15.2 Security

NFR-004: Protected functionality shall require authentication.

NFR-005: Access to functionality shall be controlled by user roles.

NFR-006: Sensitive information shall not be exposed to unauthorized users.

NFR-007: Administrative operations should be auditable.

15.3 Usability

NFR-008: The interface shall use consistent navigation.

NFR-009: Error messages shall be understandable to users.

NFR -010: Submit/Resubmit buttons shall be clear.

15.4 Reliability

NFR-011: Audit records shall be maintained reliably.

NFR-012: Past history shall not be lost.

15.5 Availability

NFR-013: Notifications shall be available.

15.6 Maintainability

NFR-014: The software shall follow a modular architecture.

NFR-015: Source code shall be maintained using version control.

NFR-016: Software documentation shall be maintained alongside relevant system changes.

15.7 Scalability

NFR-017: Architecture should allow scalability.

15.8 Compatibility

NFR-018: The system should operate correctly on commonly used modern web browsers.

15.9 Privacy

NFR-019: Personal information shall be accessible only to authorized users.

15.10 Auditability

NFR-020: Significant administrative actions should be recorded to support accountability and troubleshooting.

16. Data Requirements

The initial logical information requirements include the following entities. Data will be stored in a DBMS

User

Possible attributes:

- User ID
- Name
- Email
- Contact information
- Role
- Account status

Audit

Possible attributes

- Audit ID
- Audit coordinator

- Audit team's email address
- Date
- Time
- Criteria
- Title
- Short summary
- Faculty
- Faculty's email address
- Status

Faculty

Possible attributes

- Name
- ID
- Email
- Department
- Number of resubmissions
- Audit IDs

Audit submission

Possible attributes

- Title
- Audit ID
- Time
- Faculty
- Faculty ID
- Admin ID
- Department ID

Audit resubmission

Possible attributes

- Title
- Audit ID
- Version
- # of total comments
- Faculty ID
- Department ID

Comments

Possible attributes

- Comment ID
- Audit ID
- Title
- Summary
- Faculty ID
- Name of commentor
- Time

Audit team

Possible attributes

- Coordinator
- Coordinator ID
- Number of members
- Number of audits handled
- Emails

Notification

Possible attributes

- Notification ID
- Recipient
- Notification type
- Message
- Timestamp

Reports

Possible attributes

- Report ID
- Time
- Summary
- Department ID
- Status
- Coordinator ID

Status

Possible attributes

- Status ID
- Summary

Approval/Closure

Possible attributes

- Approval ID
- Summary
- Audit coordinator

Administrator

Possible attributes

- Admin ID
- Name
- Email

18. Major Use Cases

The initial use-case catalogue is as follows:

ID	Use case	Primary actor
UC-001	Login	All users
UC-002	Manage Users	Administrator
UC-003	Create Audit	Audit team
UC-003	Comment	Any user
UC-004	Audit Submission	Faculty
UC-005	Notify	All users
UC-006	Audit Resubmission	Faculty
UC-007	Generate Reports	Administrator/Audit team/Faculty
UC-008	Make closure reports	Audit team/Administrator
UC-009	Close audits	Audit team/Administrator
UC-010	Make Audit Report	Faculty

19 Detailed Use Cases

UC-003 – Comment

Primary Actor: Any user

Preconditions:

- Administrator/Faculty/Audit team member is authenticated.
- Audit has been published.
- The faculty has seen an audit.
- The comment button is working.

Main Flow:

1. User selects an audit.
2. System displays audit details, audit summary.
3. User types a comment message in the comment box, always open.
4. User clicks on the "Comment" button.
5. System shows comment on the audit to all the users.
6. System stores the comment in its DBMS.
7. System shows the comment to the commentor, a pop up which says "Commented!"

Alternative Flows:

- If, DBMS crashes → user's comment is stored in the filesystem.
- If, the comment button is not working → the system notifies it to user instantly or after a basic delay.
- If, the comment box is not working → system allows the user to upload a file.

Postcondition: Comments are shown.

UC-005 – Notify

Primary Actor: All users.

Preconditions:

- Administrator/Faculty/Audit team member is authenticated.
- User has selected an audit.
- User comments, raises questions, etc.
- Notification system works.

Main flow:

1. User selects an audit.
2. System displays audit details, audit summary.
3. User comments/resubmits.
4. Notification gets placed in a "bell" icon.

Alternative Flows:

- If, notify feature isn't working → system does emailing.
- If, 2 or more notifications are sent for the same comment/resubmission → system only shows 1 notification in a "bell" icon.

Postcondition: Users are notified.

UC-009 – Close audits

Primary actor: Audit team/Administrator

Preconditions:

- Administrator/Faculty/Audit team member is authenticated.
- Audits are satisfied.
- "Close" button works.
- "Users" are notified if an audit is closed basically.

Main Flow:

1. User selects an audit.
2. Audit's is satisfied.
3. User clicks on the "Close" button.
4. Users get notified using the "bell" icon basically.
5. If, users want to, they comment.

Alternative Flows:

- If, close button is not working → user is notified.
- If, close button isn't working, user wants to notify all → all are notified.
- If, comments are not working → users are notified.

Postconditions: Audits are closed.

20 User Stories

The requirements will also be represented in Agile form.

US-01

As a Faculty member, I want to log in using my credentials, so that I can safely access the audit related tasks assigned to me. FR-001

US-02

I am a Faculty member and want to provide evidence documents (PDF/DOC/images) for an audit criterion to ensure that the audit team can evaluate my department meets the criterion. FR-017, FR-018

US-03

As a Faculty member I would like to see if my submitted audit documents have been reviewed, so I can determine if further action is required on my part.
FR-021

US-04

As a Faculty member, I want to be able to resubmit audit documents after they have been rejected and, without losing the submission history, be able to correct the documents where there are deficiencies. FR-020

US-05

As a Faculty member, I would like to get a pop-up notice when the admin resubmits an audit so I will know and can reply on time. FR-28, BR-005

US-06

As an Audit Team member (Auditor), I wish to establish auditing criteria through categories of evidence requirements for departments to submit what is being audited. FR-013, FR-014

US-07

I want to be part of the Audit Team and review evidence submitted, and append Accepted, Rejected or Revision Required to the evidence to provide departments with positive or negative feedback on compliance. FR-019, FR-021

US-08

As an Audit Team member, I wish to record a compliance evaluation for each criterion (Compliant/Partially Compliant/Non Compliant/Not Applicable) in order to have a consistent audit outcome. FR-022, FR-023

US - 09

As Audit Team member, I would like to raise an Observation and add the severity level if there is evidence of gap so that the concerned department is formally notified of the gap. FR-024, FR-025

US - 10

As an Audit Team member, I want to review and approve or reject a corrective action submitted to close an observation so that observations are only closed once they are truly fixed. FR-032, FR-034

US - 11

As HOD, I want to see a schedule of the audits and criteria to plan evidence collection ahead of time. FR-013 (view), Sec 11.3

US-12

As an HOD I wish to direct responsibility for corrective action to a particular faculty member in order that there is clarity as to who is accountable for the correction of an observation. FR-030

US-13

I am an HOD and would like to see the number of items that were closed correctly in a corrective action in my department on a dashboard, to view overall audit readiness at a glance. FR-039

US - 14

As an Administrative Staff member, I want to create user accounts and assign roles, so that each user only has access to those functions that are relevant to their responsibilities. FR-004, FR-005

US-15

As an Administrative Staff member, I want to get a consolidated administrative report of system activity so that I can keep an eye on the overall usage of the portal and possible anomalies in the usage. FR-047

US-16

As a Management user, I want to see an overall dashboard with the following metrics for institutional audit health: Total audits, Open observations, Overdue actions (by department). FR-41

US-17

As an authenticated user, I want to add a comment to an audit, so that I can pose a question or clarification directly related to the context of the audit. FR-027, BR-004, BR-006

US-18

I want to automatically mark corrective actions for action date passed to be overdue, but not closed so that overdue items are visible without having to do manual tracking. FR-036

Acceptance Criteria:

US-10 – Approve or reject a corrective action.

ID	Acceptance criterion
AC-01	User is authenticated
AC-02	Audit is submitted
AC-03	Users are notified
AC-04	Evidence is present
AC-05	User has an option or a choice to choose between “accept” and “reject”
AC-06	Other parties are notified

US - 13 – View number of items closed correctly

ID	Acceptance criterion
AC-01	User is authenticated
AC-02	Admin team has the option to “close” items
AC-03	Users are notified
AC-04	Dashboard available
AC-05	Button/tab to view numbers
AC-06	View history

US - 16 – Management users see overall dashboard

ID	Acceptance criterion
AC-01	User authenticated

AC-02	Dashboard available
AC-03	View history
AC-04	Button/tab to view reports
AC-05	Merged to view different dept wise dashboards

22. Requirements Prioritization

Must Have

- Authentication
- Role-based access
- Notifications
- Comments
- "Bell" icon
- Dashboards
- Submit buttons
- Resubmit buttons

Should Have

- History accesses
- AI based comment summaries
- Criteria generation by AI
- List of criteria
- List of faculty

Could have

- Analytics
- Option for faculty to view other faculty's audits

Won't have

- Notifications to initiate process of audits

- AI based autofill in reporting

23. Requirements Traceability

Requirements Traceability Matrix (RTM).

An initial version:

Requirement	User Story	Use Case	Test
FR-010	US-006	UC-003	TBD
FR-020	US-004	UC-010	TBD
FR-011	US-008	UC-005	TBD
FR-029	US-009	UC-013	TBD
FR-034	US-013	UC-014	TBD

24. Requirements Validation

Correctness:

So, academic audit portal will make audit process faster, so it is very important.

Unambiguity:

It is completely unambiguous, as audit criteria is defined by audit teams, faculty follows criteria, comments are present to remove any confusions which arise.

Completeness:

It conveys full information, notify feature helps.

Consistency:

It doesn't conflict with any different requirements, audits happen year after year and faculty know how to handle audits.

Verifiability:

Testing team will test, UML based testing may be done as SRS is completed.

Feasibility:

It can be deployed over the internet or as an app.

Traceability:

One can trace it, use the "What's new" catalogue in the documentation too.

Prioritization:

It is important as it saves past history of audits and it is easy to use, has documentation, security is more.

Modifiability:

As modular structure is not used right now, modifiability is less, but in future plans are to use packages, modules, so it will be ok.

25. Example of Requirement Quality

Poor

The Academic Audit Portal should be fast and user-friendly for auditors, faculty and HODs.

This is ambiguous and difficult to verify. It does not say which operations should be fast, what "user-friendly" means, or how either would be tested.

Better

The system shall provide consistent navigation across the major user interfaces (the Faculty, Audit Team, HOD, Administrator and Management dashboards and the audit, evidence, observation and corrective action screens) and shall respond within acceptable time to normal user operations, such as login, evidence upload and dashboard loading, under expected workload, e.g., many faculty members submitting evidence close to an audit deadline (NFR-008, NFR-001, NFR-003).

26. Requirement Change Management

Because the Academic Audit Portal uses Agile + Incremental development, requirements can change during development. For example:

Change Request: Send automatic reminder notifications to the responsible faculty 3 days before a corrective action's due date.

The team should follow:

Change Request → Impact Analysis → Requirement Update → Backlog Update → Prioritization → Sprint Planning → Design Change → Implementation → Testing → Release

For the example above, the impact analysis would cover FR-036 (Overdue Actions), UC-005 (Notify), US-18 and the notification "bell" icon behaviour.

The change should also update relevant documentation and traceability (SRS, user stories, use cases and the Requirements Traceability Matrix).

27. Future Requirements

Potential future requirements include:

- integration with external accreditation portals (e.g., NAAC/NBA submission systems);

- automated plagiarism and document-authenticity verification of submitted evidence;
- advanced analytics and AI-based risk prediction for audit findings;
- AI-based autofill in audit reporting;
- dedicated mobile application;
- calendar integration for audit schedules and corrective action deadlines;
- automatic notifications to initiate audit cycles;
- integration with institutional authentication (single sign-on);
- integration with institutional ERP;
- multi-institution/federated audit management.

28. SRS Baseline and Versioning

- Initial baseline: SRS v1.0
- Version Control Information: SRS v1.0 covers authentication, role-based access, audit and criteria management, evidence submission and review, observations, corrective actions with closure tracking, notifications, dashboards and reports.

Contributions:

Dhruv Shanker Dandowtiya: 2nd half FRs, Data Requirements,Requirements Prioritization, rest everything.

Daksh Subhash Jain: User Stories, Requirements validation.

Aayana Jain: Traceability Matrix, 1st half FRs, Business Rules

Ayush Gupta: Non Functional Requirements**.**

AI USAGE POLICY – STUDENT DECLARATION (Assignment 2)

- AI tools may be used as supporting tools for this assignment. The team is responsible for the correctness, consistency and understanding of all submitted requirements.
- We have disclosed the use of AI tools, if any.
- AI-generated suggestions have been reviewed, verified and adapted by the team.
- We understand and can explain the submitted requirements, use cases and user stories.
- We have verified that the requirements are consistent with our project's scope.
- The final SRS represents our team's understanding and decisions.

AI Tool(s) Used: ChatGPT, Claude

Purpose of AI Use: To make FRs, User stories.

Team ID: T094

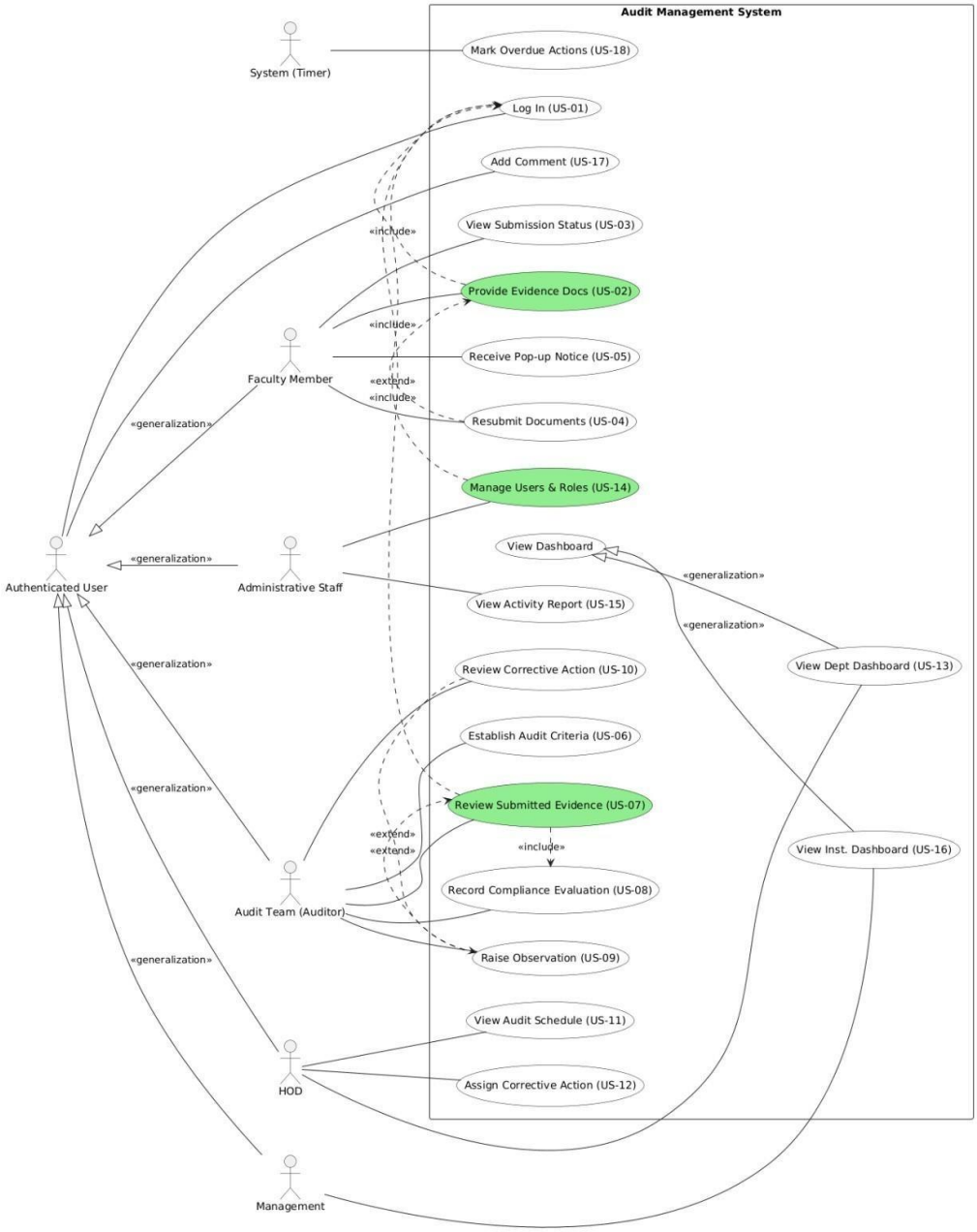
Team Representative: Dhruv Shanker Dandotiya

Date: 7 September 2026

The LNM Institute of Information Technology, Jaipur
Software Development Laboratory
Course Code: CS314
Lab Assignment 3: Solution – T094 Academic Audit Portal

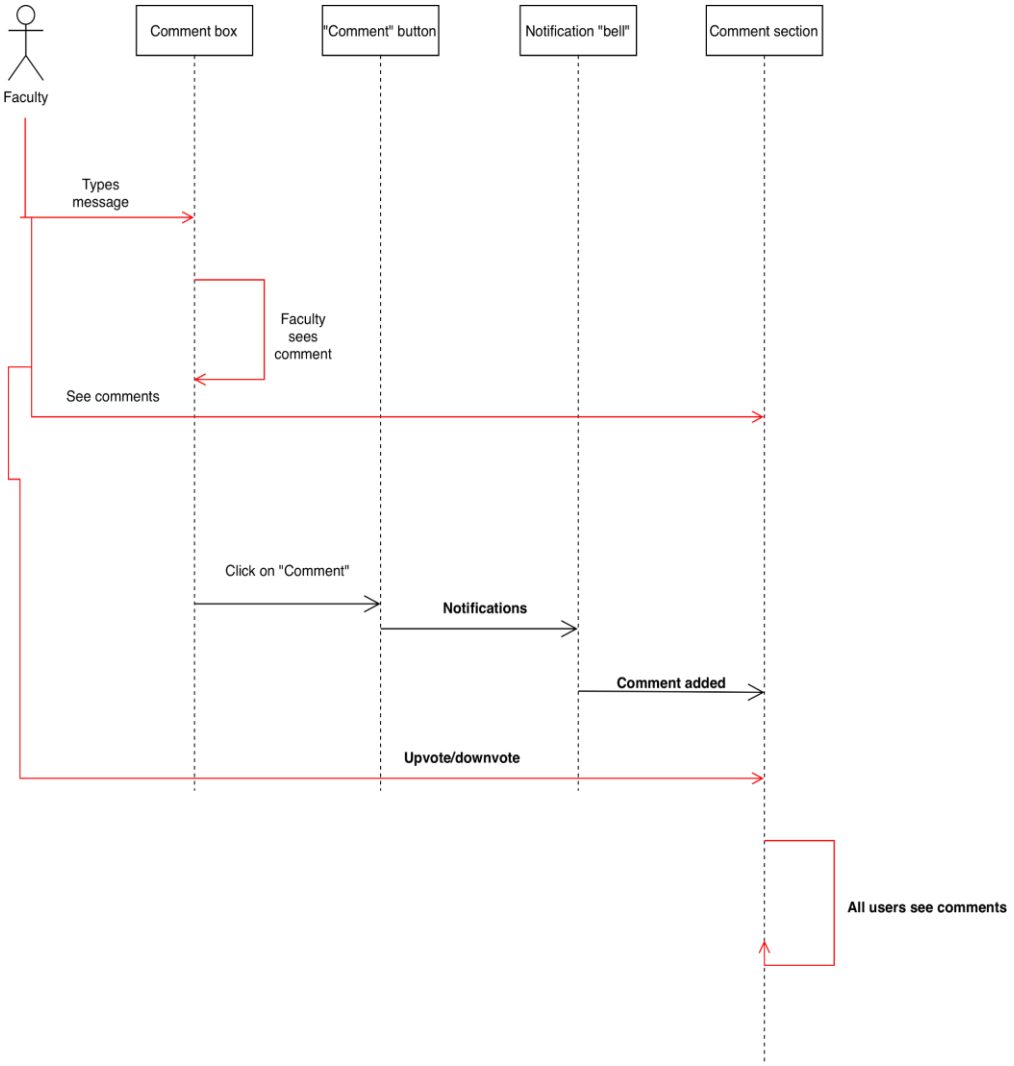
Academic Audit Portal

Use case diagram is:

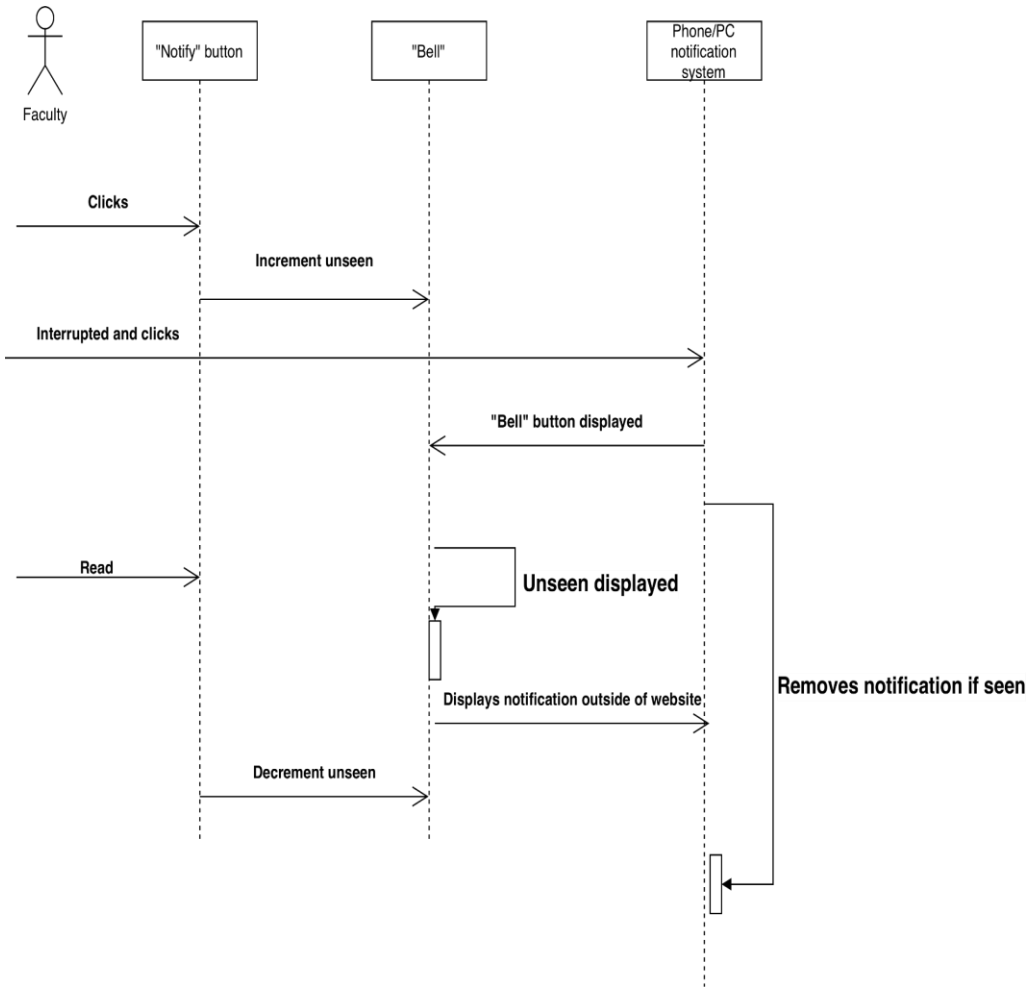


Scenario diagrams:

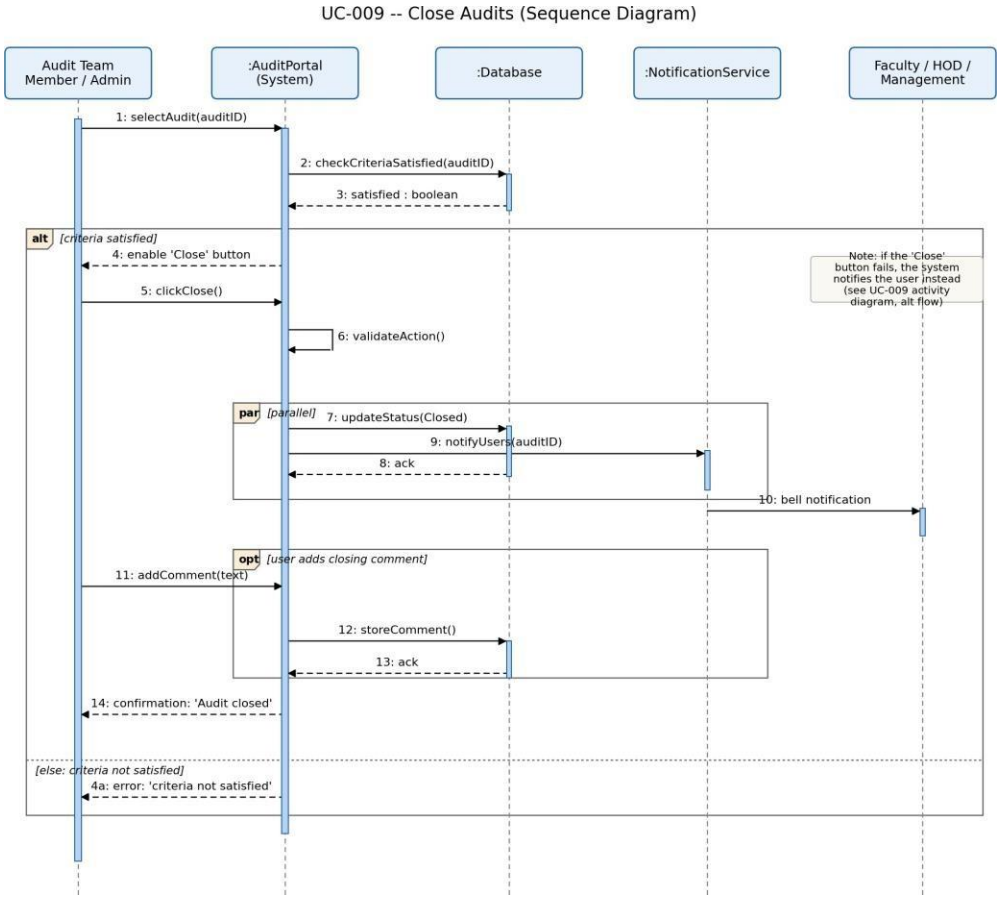
1) Comment use case is:



2) Notify use case is:



3) Close audits is:



Activities diagrams are:

1. Comment:

   Attached: “Act1-Page-1.pdf”

2. Notify:

   Attached: “Act1-Page-2.pdf”

3. Close audit:

   Attached: “Act1-Page-3.pdf”

Contributions:

LNMIIT
The LNM Institute of Information Technology

Daksh Jain: Use case diagram
Ayush Gupta: Scenario diagrams
Aayana Jain: Notify activity diagram
Dhruv Shanker Dandowtiya: Comment, Close activity diagrams

AI USAGE POLICY – STUDENT DECLARATION

AI tools may be used as supporting tools for this assignment. The team is responsible for the correctness, consistency and understanding of all submitted scenarios and UML models.

• We have disclosed the use of AI tools, if any.
• AI-generated suggestions have been reviewed, verified and adapted by the team.
• We understand and can explain the submitted scenarios, use case diagram and activity diagrams.
• We have verified that the UML models are consistent with our project's requirements, use cases and scope.
• The final submission represents our team's understanding and modeling decisions.

AI Tool(s) Used: Claude
Purpose of AI Use: Help in activity diagram, mindmaps
Team ID: T094
Team Representative: Dhruv Shanker Dandotiya
Date: 13th September 2026

LNMIIT
The LNM Institute of Information Technology

The LNM Institute of Information Technology, Jaipur
Software Development Laboratory
Course Code: CS314
Lab Assignment 4: Solution

1. Classes

The following classes have been identified to support the Academic Audit Portal based on the requirements and use cases (UC-003 Comment, UC-005 Notify, UC-009 Close audits).

User (Abstract)	
Attributes	userID name email role accountStatus
Operations	login() logout()

Faculty	
Attributes	departmentID assignedCourses
Operations	submitEvidence() addComment() viewSubmissionStatus() resubmitEvidence()

AuditCriterion	
Attributes	criterionID Name Category Requirement Priority
Operations	evaluate()

Evidence	
Attributes	evidenceID title fileType status version
Operations	upload() download() review() resubmit()

Observation	
Attributes	observationID Description Severity status
Operations	create() updateStatus() reopen()

CorrectiveAction	
Attributes	actionID Description dueDate Priority status
Operations	assign() submit() approve() checkOverdue()

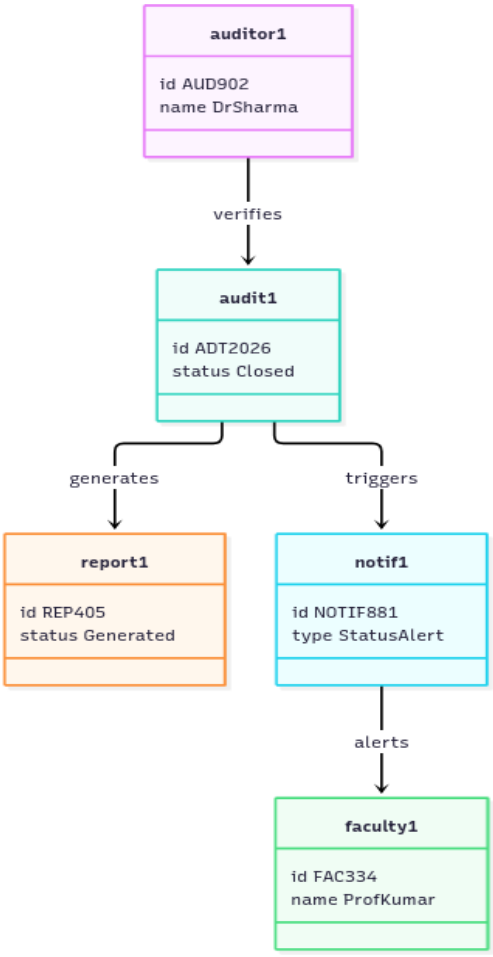
Comment	
Attributes	commented Text timestamp
Operations	store() display()

Notification	
Attributes	notificationID Recipient Message isRead Timestamp type
Operations	send() markAsRead()

AuditReport	
Attributes	reportID Summary closureDate
Operations	generate() export()

2. Object Diagram

Scenario Description (UC-009: Close audits): An Auditor object (auditor1) verifies that all AuditCriterion objects for a specific Audit object (audit1) are satisfied. The auditor updates the audit's status to closed. Consequently, an AuditReport object (report1) is generated documenting the closure. The system then creates a Notification object (notif1) to alert the associated Faculty object (faculty1) regarding the closed status.



3. Relationships

Generalization / Inheritance

- Faculty, Auditor, Administrator, HOD and Management all inherit from the abstract User class (shared login/logout per FR-001–FR-005).

Composition

- An audit is composed of AuditCriterion instances (FR-013/FR-015) — the criteria are explicitly defined for that specific audit and cannot exist independently of it.
- An audit is composed of Comments (BR-004: addition of comments must be possible). The comments cannot exist outside the context of the audit.

Association & Multiplicity

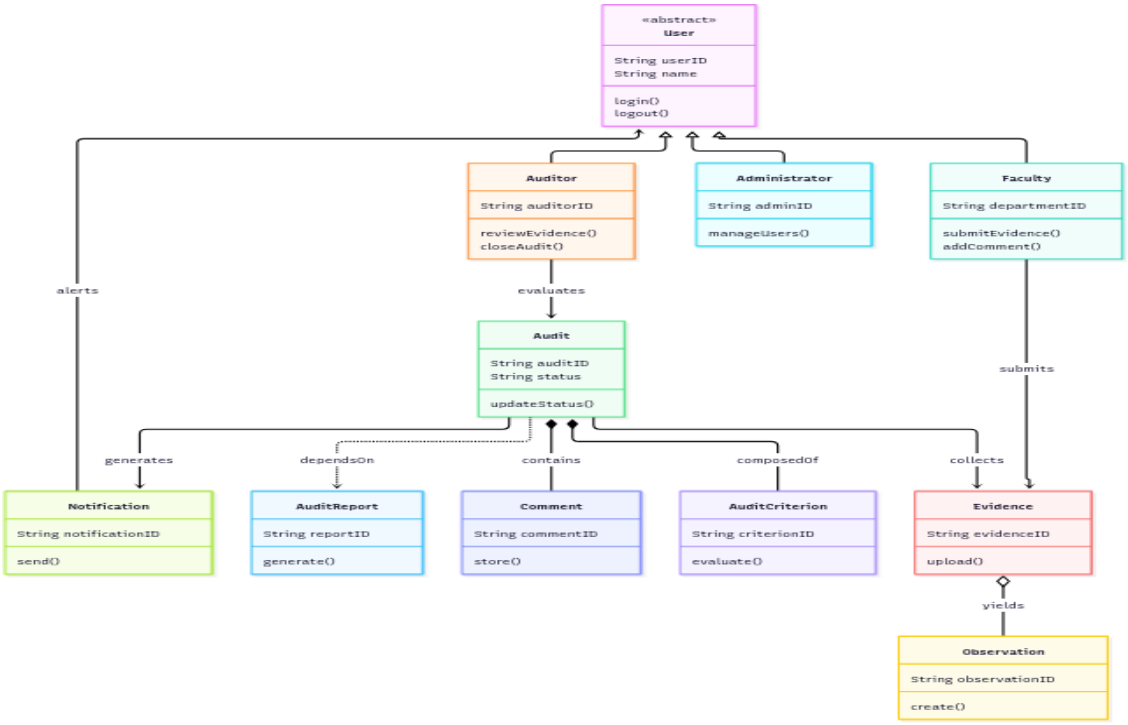
- Auditor (1..*) evaluates Audit (1..*) — FR-022 Criterion Evaluation.
- Faculty (1) submits Evidence (0..*) for an Audit (1) — FR-017/FR-018 Evidence Submission.
- Audit (1) triggers Notification (0..*) — BR-005/BR-007/BR-011/BR-012 notification rules.
- HOD (1) assigns CorrectiveAction (0..*) to Faculty (0..*) — FR-030 Assign Responsible Person.

Aggregation

- Evidence (1) aggregates Observation (0..*) — FR-024 Create Observation. Observations form a historical record tied to the evidence review, but could be reassigned if evidence is revised (FR-020 Evidence Versioning).

Dependency

- AuditReport depends on the Audit class, as the report is generated based on the final state of the audit — FR-042 Audit Report.
- CorrectiveAction depends on Observation, since an action only exists to remediate a specific observation — FR-029 Create Corrective Action, BR-010 Faculty cannot edit the audit team's criteria.
-



4. Consistency Statement

Traceability is maintained across the modelling chain: Assignment 2 Requirements → 3 Use Cases (UC-003 Comment, UC-005 Notify, UC-009 Close Audit) → Assignment 3 Activities → Classes → Objects → Relationships. Every object in the Object Diagram (auditor1, audit1, report1, notif1, faculty1) is an instance of a class defined in the Class Diagram (Auditor, Audit, AuditReport, Notification, Faculty respectively), and every relationship documented in Section 3 is justified against one of the three selected use cases.

Use Case	FR(s)	BR(s)	Classes Involved	Objects/Relationships
UC-003 Comment	FR-027	BR-004,BR-006	Faculty,Audit, Comment	Audit-contains-Comment
UC-005 Notify	FR-28	BR-005,BR-007 BR-011,BR-012	Audit, Notification, Faculty	audit1, notif1, faculty1; Audit –triggers– Notification; Notification –alerts– Faculty
UC-009 Close Audit	FR-012,FR-022 FR-033,FR-034	BR-002,BR-003	Auditor,Audit, AuditCriterion, AuditReport	auditor1, audit1, report1; Auditor –evaluates– Audit; Audit–contains–AuditCriterion; Audit –generates– AuditReport

5. Contributions

Name	Contribution
Ayush Gupta	Object Diagram
Aayana Jain	Relationships
Daksh Subhash Jain	Classes
Dhruv Shanker Dandowtiya	Consistency Statement

6. AI Usage Policy — Student Declaration

AI tools may be used as supporting tools for this assignment. The team is responsible for the correctness, consistency and understanding of all submitted classes, objects and relationships.

- We have disclosed the use of AI tools, if any.
- AI-generated suggestions have been reviewed, verified and adapted by the team.
- We understand and can explain the submitted classes, object diagram and relationships.
- We have verified that the UML models are consistent with our project's requirements, use cases and scope.
- The final submission represents our team's understanding and modeling decisions.

AI Tool(s) Used: Claude (Anthropic)

Purpose of AI Use: Assisted in identifying classes, drafting the object diagram scenario and justifying class relationships for the Academic Audit Portal, consistent with AI usage already disclosed in Assignments 1–3.

Team ID: T094

The LNM Institute of Information Technology, Jaipur
Software Development Laboratory
Course Code: CS314

Lab Assignment 5: Solution

Collaboration, State and Component Modelling

T094 – Academic Audit Portal

Item	Details
Team ID	T094
Team Members	Ayush Gupta, Aayana Jain, Daksh Jain, Dhruv Shanker Dandotiya
Roll Numbers	24ucs261, 24ucs262, 24ucs263, 24ucs264
Use cases modelled	UC-003 Comment, UC-005 Notify, UC-009 Close audits (same three detailed use cases as Assignments 2–4)
Notation	UML 1.x (collaboration, state and component diagrams)

1. Collaboration Diagrams

One UML collaboration diagram is given for each of the three detailed use cases. The participating objects are instances of, or interface/control objects for, the classes identified in Assignment 4 (User/Faculty, Audit, AuditCriterion, AuditReport, Comment, Notification, etc.) plus the DBMS from the data requirements of Assignment 2.

Reading convention used in all three diagrams:

- **No. n** – order number of a message in the main flow.
- **Alt. no. n** – order number of a message in an alternative flow.
- **Unnumbered actor links** (Press, Show, Typing facility, Send facility, Viewing facility) – the interface between the actor and a user-visible object; they are links, not part of the numbered internal sequence.

1.1 UC-003 – Comment

Scenario: an authenticated user (Faculty, Audit team member or Administrator) types a message in the comment box and clicks the Comment button. The system requests a pop-up and the database, stores the comment, notifies all users, adds the comment to the comment section of the audit and shows the "Commented!" pop-up to the commentor. Optional flows show the commentor's profile and allow upvoting/downvoting of a comment.

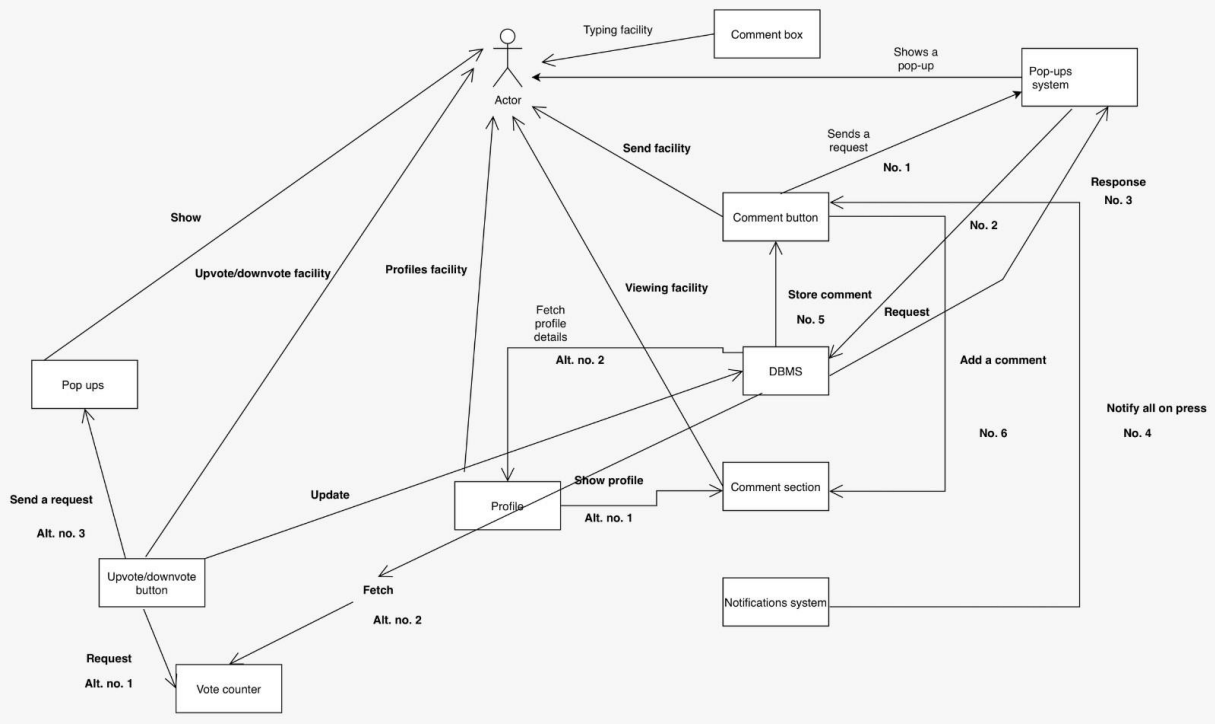


Figure 1: Collaboration Diagram – UC-003 Comment (Collaboration-Diagram-UC01.png)

Objects, class mapping and responsibilities

Object in diagram	Class (Assignment 4)	Responsibility in completing the use case
Actor	Faculty (via abstract User)	Types the comment, presses Comment, views the comment section and pop-ups.
Comment box	Comment (Text)	Provides the typing facility; captures the message text.
Comment button	Comment (store())	Control object: receives the send action and drives the whole interaction.
Comment section	Audit (composition) / Comment (display())	Shows all comments of the audit to every user (viewing facility).
DBMS	Persistent store	Stores the comment and returns responses/records (store comment, fetch profile, update votes).
Pop-ups system	Notification (type = pop-up)	Displays the “Commented!” pop-up to the commentor.
Notifications system	Notification (send())	Notifies all users when a comment is posted (“notify all on press”).
Profile	User (name, role)	Alternative flow: shows the commentor’s profile details.
Upvote/downvote button, Vote counter, Pop ups	Comment (voting, from the Assignment 3 comment scenario)	Alternative flow: records votes on a comment and confirms with a pop-up.

Messages (links and sequence)

Seq.	From → To	Message / purpose
—	Comment box → Actor	Typing facility: user types the comment.
—	Actor ↔ Comment button	Send facility: user clicks "Comment".
No. 1	Comment button → Pop-ups system	Sends a request (a pop-up is needed for this comment).
No. 2	Pop-ups system → DBMS	Request to the database.
No. 3	DBMS → Pop-ups system	Response (operation done); pop-up is then shown to the Actor.
No. 4	Notifications system → Comment button	Notify all on press.
No. 5	DBMS ↔ Comment button	Store comment (as drawn, link between DBMS and Comment button).
No. 6	Comment button → Comment section	Add a comment to the audit's comment section.
—	Comment section → Actor	Viewing facility: comment is visible to all users.
—	Pop-ups system → Actor	Shows a pop-up ("Commented!").
Alt. 1 (profile)	Profile → Comment section	Show profile next to the comment.
Alt. 2 (profile)	DBMS → Profile	Fetch profile details; Profile → Actor is the profiles facility.
Alt. 1 (vote)	Upvote/downvote button → Vote counter	Request to change the vote count.
Alt. 2 (vote)	DBMS → Vote counter	Fetch the current count; the button also sends an Update to the DBMS.
Alt. 3 (vote)	Upvote/downvote button → Pop ups	Send a request; the pop-up is shown to the Actor.

1.2 UC-005 – Notify

Scenario: when a user comments, resubmits, or a deadline passes, a notification is created. It may be triggered manually (Manual notify button, main flow) or automatically by the comment/deadline-pass/resubmission button (Alt. no. 1). The notification is stored in the DBMS, placed in the "bell" notification storage, sent to the phone/PC, and announced by a pop-up.

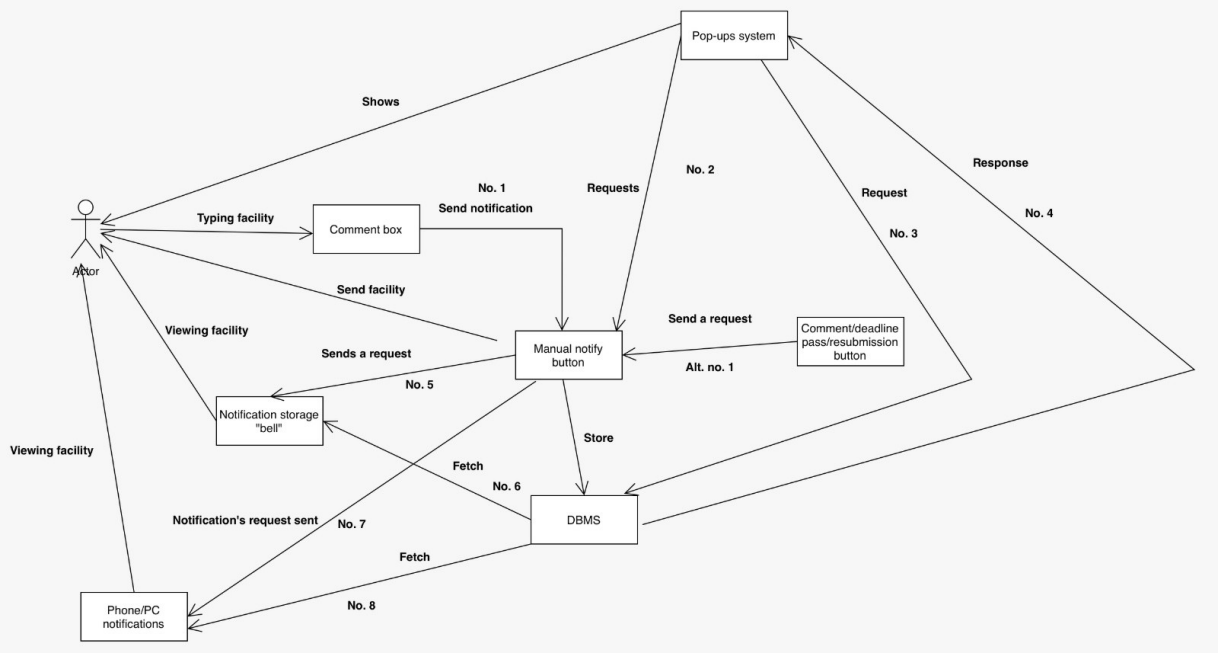


Figure 2: Collaboration Diagram – UC-005 Notify (Collaboration-Diagram-UC02.png)

Objects, class mapping and responsibilities

Object in diagram	Class (Assignment 4)	Responsibility in completing the use case
Actor	Faculty / Administrator (via User)	Types a comment or presses Notify; sees the bell, phone/PC alert and pop-up.
Comment box	Comment (Text)	Typing facility; sending the comment raises the notification (No. 1).
Comment/deadline pass/resubmission button	Comment.store(), CorrectiveAction.checkOverdue(), Evidence.resubmit()	Event source for Alt. no. 1: sends a request to notify without a manual press.
Manual notify button	Notification (send())	Control object: creates the notification, stores it and dispatches it to every channel.
Notification storage “bell”	Notification (isRead, markAsRead())	Holds unseen notifications and shows them to the Actor (viewing facility).
Phone/PC notifications	Notification (type)	Delivers the alert outside the website.
Pop-ups system	Notification (type = pop-up)	Requests/receives data from the DBMS and shows a pop-up to the Actor.
DBMS	Persistent store	Stores the notification and supplies it to the bell and phone/PC alert.

Messages (links and sequence)

Seq.	From → To	Message / purpose
—	Actor → Comment box	Typing facility.
No. 1	Comment box → Manual notify button	Send notification.
Alt. no. 1	Comment/deadline pass/resubmission button → Manual notify button	Send a request (automatic trigger).
—	Actor ↔ Manual notify button	Send facility (manual press).
—	Manual notify button → DBMS	Store the notification.
No. 2	Pop-ups system → Manual notify button	Requests notification data.
No. 3	Pop-ups system → DBMS	Request.
No. 4	DBMS → Pop-ups system	Response; pop-up is shown to the Actor.
No. 5	Manual notify button → Notification storage “bell”	Sends a request to place the notification in the bell.
No. 6	DBMS → Notification storage “bell”	Fetch; bell is viewed by the Actor.
No. 7	Manual notify button → Phone/PC notifications	Notification’s request sent.
No. 8	DBMS → Phone/PC notifications	Fetch; phone/PC alert is viewed by the Actor.

1.3 UC-009 – Close audits

Scenario: the audit team member/administrator presses the Close audits button once the audit is satisfied. The closure is added to the DBMS, a pop-up is requested, the notification system is asked to notify users, and the audit status value and audit closure date are requested, fetched from the DBMS and shown to the actor.

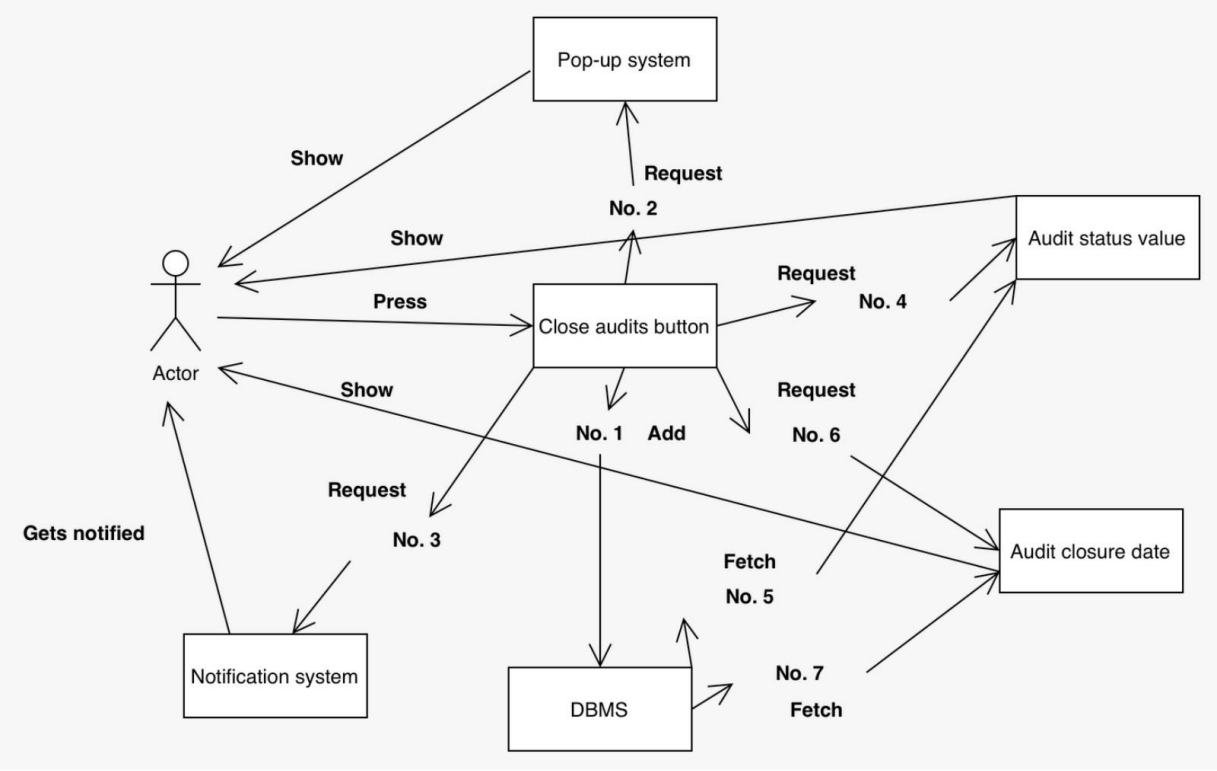


Figure 3: Collaboration Diagram – UC-009 Close audits (Collaboration-Diagram-UC03.png)

Objects, class mapping and responsibilities

Object in diagram	Class (Assignment 4)	Responsibility in completing the use case
Actor	Auditor / Administrator (via User)	Presses "Close"; sees the pop-up, status, closure date and notification.
Close audits button	Auditor (closeAudit()) / Audit (updateStatus())	Control object: coordinates all requests needed to close the audit.
Audit status value	Audit (status)	Holds and shows the new status (Closed).
Audit closure date	AuditReport (closureDate)	Holds and shows the closure date documented in the report.
Pop-up system	Notification (type = pop-up)	Shows the "audit closed!" pop-up.
Notification system	Notification (send())	Notifies users through the bell (Faculty/HOD/Management).
DBMS	Persistent store	Adds the closure record and supplies status and closure date.

Messages (links and sequence)

Seq.	From → To	Message / purpose
—	Actor → Close audits button	Press.
No. 1	Close audits button → DBMS	Add the closure to the database.

Seq.	From → To	Message / purpose
No. 2	Close audits button → Pop-up system	Request; the pop-up is shown to the Actor.
No. 3	Close audits button → Notification system	Request; Notification system notifies the Actor (“gets notified”).
No. 4	Close audits button → Audit status value	Request the status; shown to the Actor.
No. 5	DBMS → Audit status value	Fetch the stored status.
No. 6	Close audits button → Audit closure date	Request the closure date; shown to the Actor.
No. 7	DBMS → Audit closure date	Fetch the stored closure date.

2. State Diagrams

Two important entities whose state changes across the selected use cases were chosen: the Comment (UC-003 – its query/storage life cycle) and the Notification (UC-005, and also UC-003/UC-009 which raise notifications).

2.1 State Diagram 01 – Comment (query life cycle)

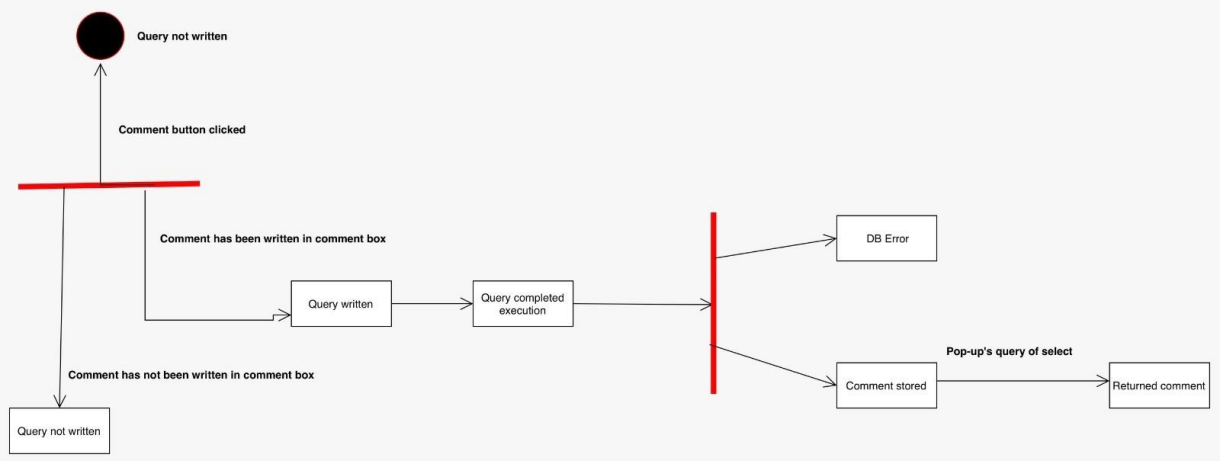


Figure 4: State Diagram – Comment (State-Diagram-01.png)

Element	Description
Initial state	Query not written (comment not yet typed/submitted).
Intermediate states	Query written; Query completed execution; Comment stored.
Events	Comment button clicked; comment written / not written in the comment box; query executed; DB responds (success or failure); pop-up select query.
Final states	Returned comment (successful path). DB Error (failure path).

From state	Event [guard]	To state
Query not written (initial)	Comment button clicked	Decision on the comment box
Decision	[Comment has not been written in comment box]	Query not written
Decision	[Comment has been written in comment box]	Query written
Query written	Query executes	Query completed execution
Query completed execution	[DB failure]	DB Error
Query completed execution	[DB success]	Comment stored
Comment stored	Pop-up's query of select	Returned comment

2.2 State Diagram 02 – Notification

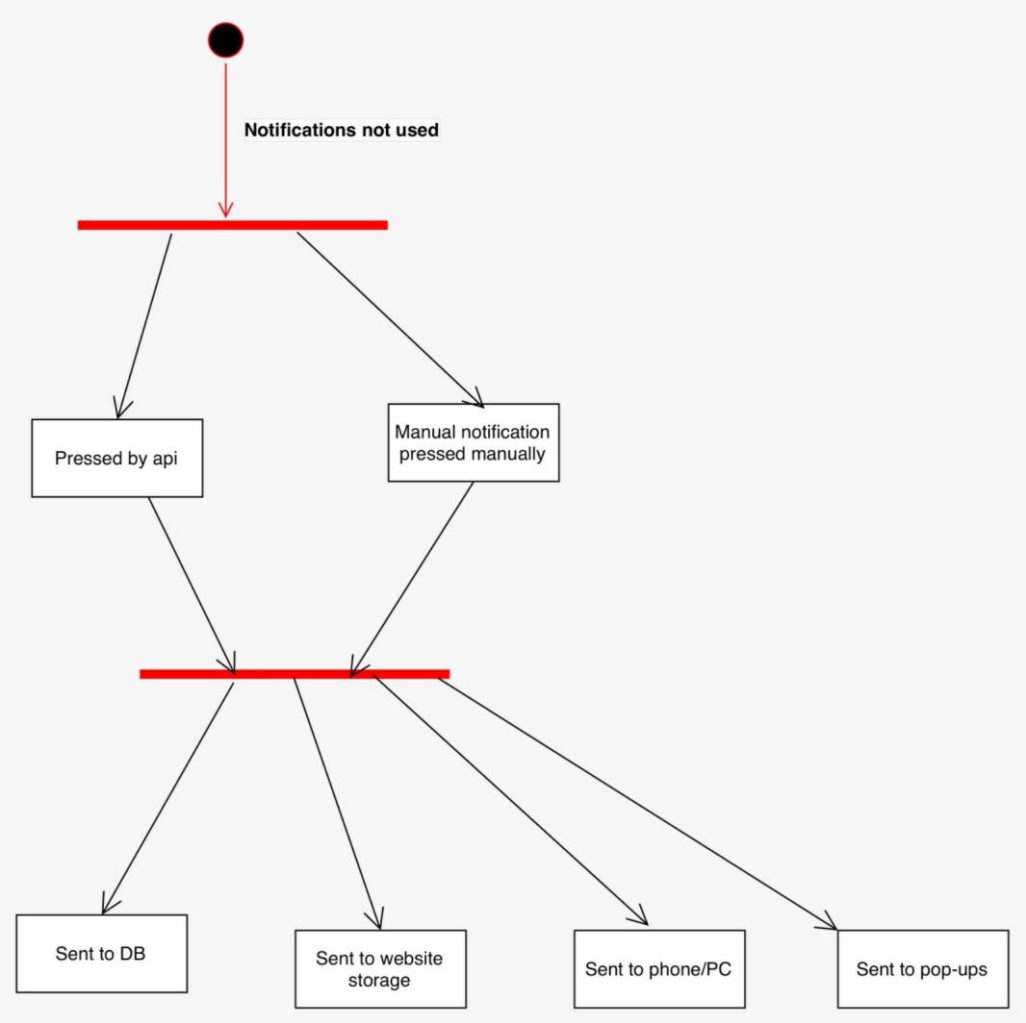


Figure 5: State Diagram – Notification (State-Diagram-02.png)

Element	Description
Initial state	Notifications not used (no notification raised yet).
Intermediate states	Pressed by API (automatic trigger – comment, deadline pass, resubmission); Manual notification pressed manually (Manual notify button).
Events	API trigger; manual press of the notify button; dispatch to the four channels.
Guards / conditions	Exactly one of the two trigger sources starts a notification (fork); both paths join before dispatch.
Final states	After the join, the notification is dispatched concurrently and ends in four states: Sent to DB, Sent to website storage (bell), Sent to phone/PC, Sent to pop-ups.

From state	Event [guard]	To state
Notifications not used (initial)	Notification needed	Trigger selection (fork)
Trigger selection	[raised by API / system event]	Pressed by API
Trigger selection	[raised by user pressing Notify]	Manual notification pressed manually
Pressed by API	Trigger accepted	Dispatch (join, then fork)
Manual notification pressed manually	Trigger accepted	Dispatch (join, then fork)
Dispatch	Concurrent send	Sent to DB
Dispatch	Concurrent send	Sent to website storage
Dispatch	Concurrent send	Sent to phone/PC
Dispatch	Concurrent send	Sent to pop-ups

3. Component Diagram

The system is organised as a layered, modular web application (NFR-014): UI components, provided interfaces, manager (business-logic) components, a data access layer and external services. The pop-up system is a shared UI service used by the Comment, Notification and Audit UIs.

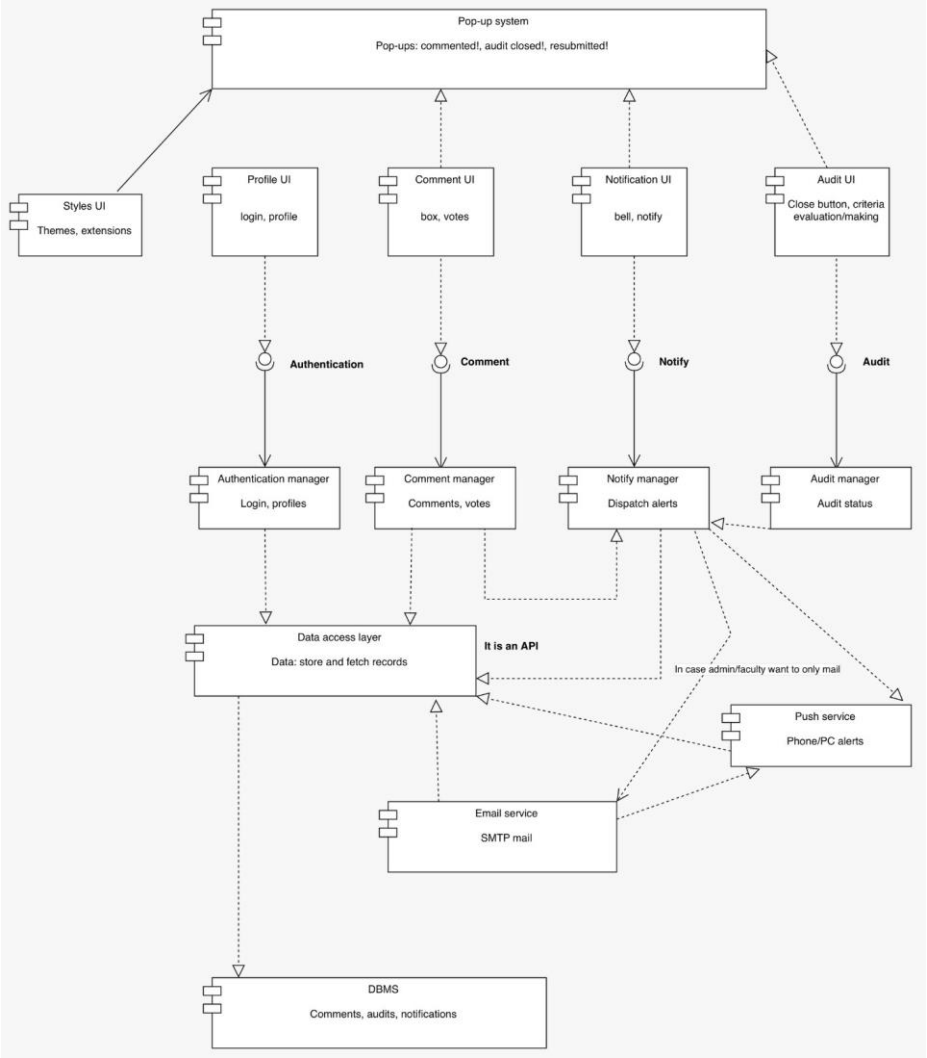


Figure 6: Component Diagram – Academic Audit Portal (Component-Diagram.png)

3.1 Components

Component	Layer	Responsibility
Pop-up system	UI service	Shows the pop-ups “commented!”, “audit closed!” and “resubmitted!”.
Styles UI	UI	Themes and extensions; styles the pop-up system.
Profile UI	UI	Login and profile screens.
Comment UI	UI	Comment box and votes.
Notification UI	UI	Bell and notify button.
Audit UI	UI	Close button and criteria evaluation/making.
Authentication manager	Business logic	Login and profiles.
Comment manager	Business logic	Comments and votes.

Component	Layer	Responsibility
Notify manager	Business logic	Dispatches alerts to the bell, push and e-mail channels.
Audit manager	Business logic	Audit status (closing an audit).
Data access layer	Data (API)	Stores and fetches records for all managers and services.
Push service	External-facing service	Phone/PC alerts.
Email service	External-facing service	SMTP mail; used when admin/faculty want to only mail (and as fallback when notify fails, UC-005).
DBMS	Data store	Stores comments, audits and notifications.

3.2 Interfaces

Interface	Required by (client)	Provided by (server)	Operations covered
Authentication	Profile UI	Authentication manager	login, profile (User.login/logout)
Comment	Comment UI	Comment manager	add comment, vote (Comment.store/display)
Notify	Notification UI	Notify manager	send notification, bell (Notification.send/markAs Read)
Audit	Audit UI	Audit manager	close audit, audit status (Audit.updateStatus)

3.3 Dependencies and relationships

- Styles UI uses the Pop-up system; the Comment UI, Notification UI and Audit UI each depend on the Pop-up system to show their pop-ups.
- Comment manager depends on the Notify manager (notify all when a comment is added); Audit manager depends on the Notify manager (notify users when an audit is closed).
- Authentication manager, Comment manager and Notify manager depend on the Data access layer.
- Notify manager depends on the Push service and the Email service; Push service and Email service depend on the Data access layer, and the Email service also relates to the Push service.
- Data access layer depends on the DBMS (single point of persistence).

3.4 External systems/services

External system/service	Role
DBMS	Centralised database for comments, audits and notifications (Assignment 1 assumption: centralised database).
SMTP mail server (Email service)	Delivers e-mail notifications.

External system/service	Role
Phone/PC alert channel (Push service)	Delivers alerts outside the website.

3.5 Use case to component realization

Use case	Components that realize it
UC-003 Comment	Comment UI → Comment interface → Comment manager → Data access layer → DBMS; Profile UI/Authentication manager (commentor profile); Pop-up system ("commented!"); Notify manager (notify all).
UC-005 Notify	Notification UI → Notify interface → Notify manager → Push service, Email service, Data access layer → DBMS; Pop-up system ("resubmitted!").
UC-009 Close audits	Audit UI → Audit interface → Audit manager → Notify manager → Data access layer → DBMS; Pop-up system ("audit closed!").

4. Consistency Statement

Traceability is maintained across the complete modelling chain: Assignment 2 Requirements → 3 Use Cases (UC-003 Comment, UC-005 Notify, UC-009 Close audits) → Assignment 3 Scenarios and Activities → Assignment 4 Classes → Objects/Relationships → Assignment 5 Collaboration → States → Components. Every participating object in the collaboration diagrams is either an instance of a class from the Assignment 4 class diagram or a user-interface/control object that realises an operation of such a class, as mapped in Section 1.

Use case	FR / BR	Assignment 4 classes	Collaboration objects	State diagram	Components
UC-003 Comment	FR-027; BR-004, BR-006; US-17	Faculty, Audit, Comment	Comment box, Comment button, Comment section, DBMS, Pop-ups system, Notifications system	State-Diagram-01 (Comment)	Comment UI, Comment manager, Pop-up system, Data access layer, DBMS
UC-005 Notify	FR-28; BR-005, BR-007, BR-011, BR-012; US-05	Audit, Notification, Faculty	Manual notify button, bell, Phone/PC notifications, Pop-ups system, DBMS	State-Diagram-02 (Notification)	Notification UI, Notify manager, Push service, Email service, DBMS
UC-009 Close audits	FR-012, FR-022, FR-033, FR-034; BR-002, BR-003	Auditor, Audit, AuditCriterion, AuditReport	Close audits button, Audit status value, Audit closure date, Notification system, DBMS	State-Diagram-02 (Notification raised on closure)	Audit UI, Audit manager, Notify manager, DBMS

Modelling decisions to note:

- The bell, pop-ups and phone/PC alerts are delivery channels of the Notification class (attribute type); the Audit status value and Audit closure date correspond to Audit.status and AuditReport.closureDate.
- The Profile, Upvote/downvote button and Vote counter come from the Comment scenario of Assignment 3 and are treated as extensions of User and Comment; they are shown only in alternative flows.
- The Data access layer and Email service in the component diagram support the alternative flows of UC-003 (DBMS failure) and UC-005 (notify feature not working → e-mail).

5. Contributions

Name	Contribution
Ayush Gupta	Collaboration Diagrams – UC-003 Comment
Aayana Jain	Collaboration Diagrams – UC-005 Notify and Notification State Diagram
Daksh Subhash Jain	Comment State Diagram and Consistency Statement
Dhruv Shanker Dandotiya	Collaboration Diagrams – UC-009 Close Audits and Component

6. AI Usage Policy — Student Declaration

AI tools may be used as supporting tools for this assignment. The team is responsible for the correctness, consistency and understanding of all submitted collaboration, state and component models.

- We have disclosed the use of AI tools, if any.
- AI-generated suggestions have been reviewed, verified and adapted by the team.
- We understand and can explain the submitted collaboration diagrams, state diagrams and component diagrams.
- We have verified that the UML models are consistent with our project's requirements, use cases, classes, objects and scope.
- The final submission represents our team's understanding and modeling decisions.

AI Tool(s) Used: Claude (Anthropic)

Purpose of AI Use: Assisted in structuring this document, drafting the object/message tables and the consistency mapping for the Academic Audit Portal, consistent with AI usage already disclosed in Assignments 1–4.

Team ID: T094

Team Representative: Dhruv Shanker Dandotiya

Date: 29 September 2026

The LNM Institute of Information Technology, Jaipur
Software Development Laboratory
Course Code: CS314

Lab Assignment 6: Solution

Software Design Modelling
T094 – Academic Audit Portal

Item	Details
Team members	Ayush Gupta, Aayana Jain, Daksh Jain, Dhruv Shanker Dandotiya
Roll numbers	24ucs261, 24ucs262, 24ucs263, 24ucs264
Selected use cases	UC-003 Comment; UC-005 Notify; UC-009 Close audits

1. Objective

To transform the Academic Audit Portal analysis models into an implementable software design. Assignment 6 covers data/class design and architectural design. Assignment 7 continues the same model with user interfaces, software interfaces, component logic and end-to-end traceability.

2. Data/Class Design

2.1 Refinement approach

The User hierarchy, Audit, AuditCriterion, Evidence, Observation, CorrectiveAction, Comment, Notification and AuditReport remain the domain model. Private typed attributes, public operation signatures, role checks, persistent keys and transaction boundaries refine these analysis classes. The managers and data access layer already identified in Assignment 5 become implementation classes.

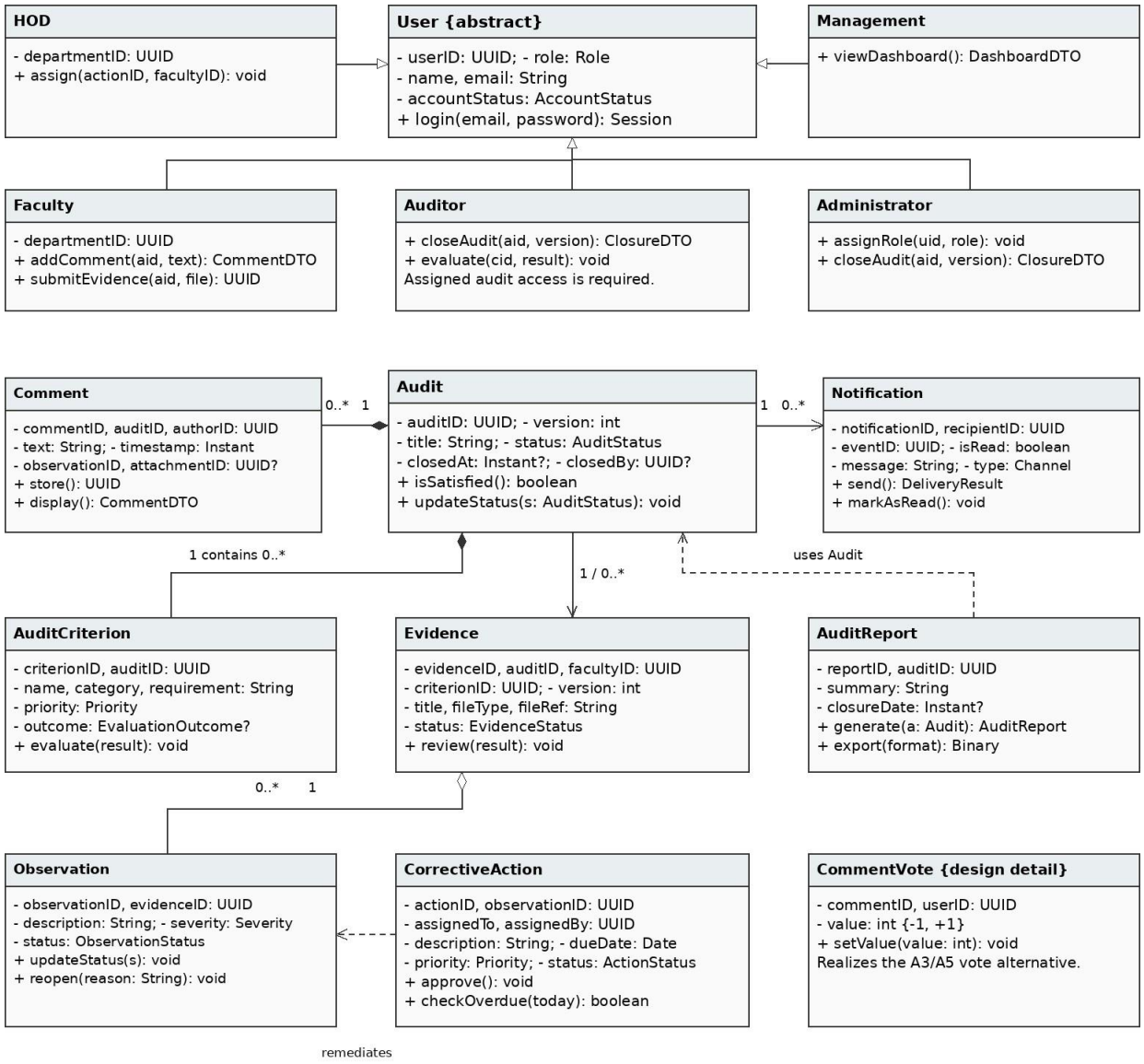
UI objects such as buttons, the bell and pop-ups remain presentation elements; they do not become persistent domain entities. A durable pending-comment store implements the existing database-failure alternative, and notification event identifiers implement the existing duplicate-suppression alternative.

2.2 Baseline interpretation

The source catalogue uses UC-003 for both Create Audit and Comment. Here UC-003 always means Comment, matching the three detailed use cases in Assignments 2–5. The existing requirement spelling FR-28 is retained. “All users” means users authorized to access that audit, consistent with FR-005 and NFR-006. No new business feature is introduced.

2.3 UML Design Class Diagram

Figure 1 is one design model presented in two readable views. Figure 1a contains domain classes and relationships; Figure 1b contains implementation classes and dependencies. Complete attribute and operation specifications follow. The combined diagram is included in the editable source package.



Associations retained from A4

Auditor 1..* — 1..* Audit (active audits); Faculty 1 — 0..* Evidence.
HOD 1 — 0..* CorrectiveAction; each assigned action has one responsible Faculty.
User 1 — 0..* Comment (author); User 1 — 0..* Notification (recipient).
Audit 1 — 0..* AuditReport; Comment 1 — 0..* CommentVote.
Filled diamond: composition. Hollow diamond: aggregation. Triangle: inheritance.
Dashed arrow: dependency. + public; - private; ? nullable. Detailed signatures follow.

Figure 1a: Domain view of the UML Design Class Diagram

2.4 Implementation Classes in the Design Model

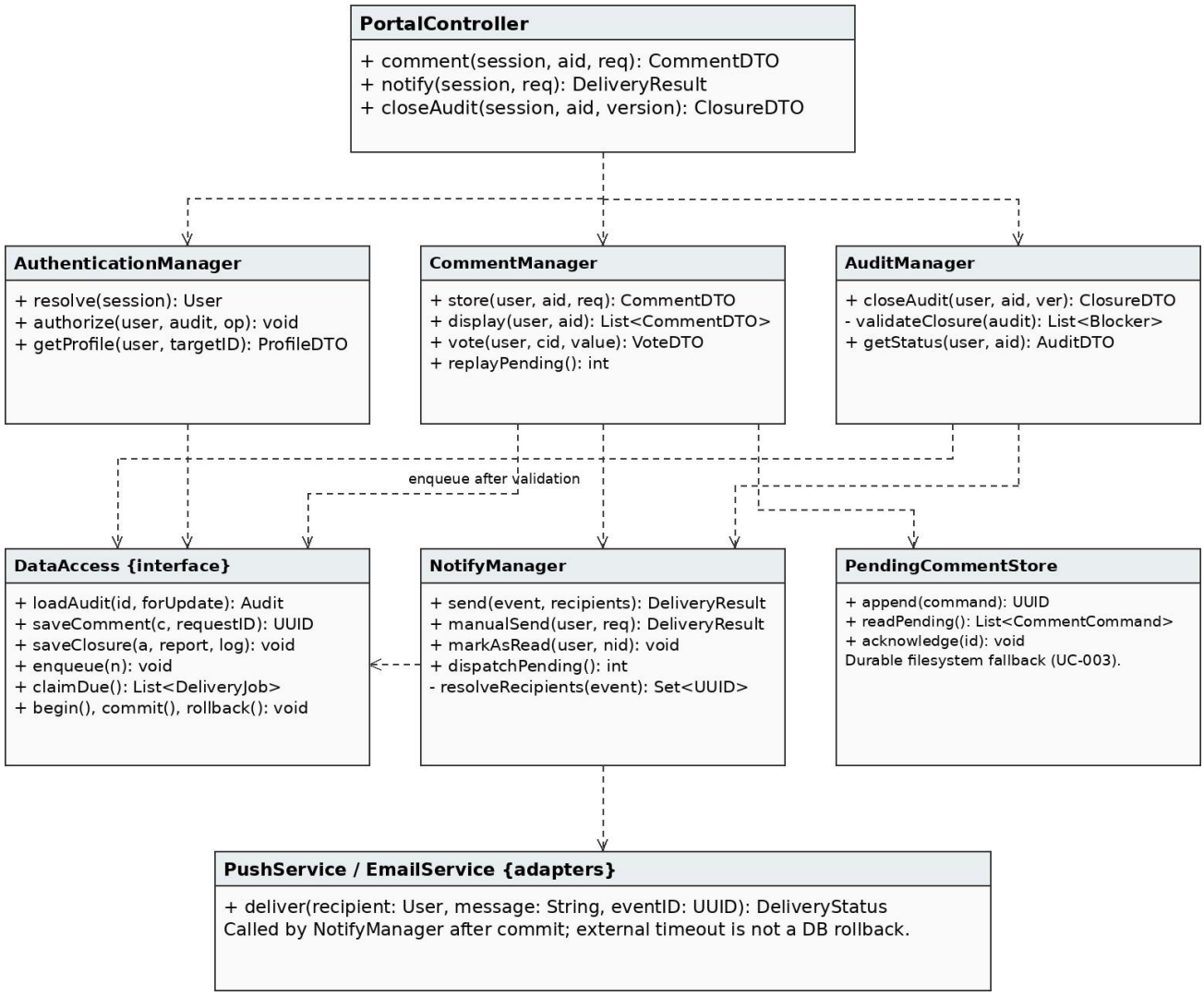


Figure 1b: Application and persistence view of the same design model

PortalController delegates to the A5 managers. User actor operations are realized through these managers with the authenticated User as a parameter. Managers manipulate the domain classes in Figure 1a and persist them through DataAccess. Dependency arrows point from client to supplier. DataAccess is implemented by a DBMS adapter; it is not a second database.

Signatures in the diagrams abbreviate identifiers as aid (auditID), cid (commentID), nid (notificationID) and ver (expectedVersion). UUID is the identifier type; Instant is a UTC timestamp; Date is a calendar date; Binary is file content. No caller-provided role or userID is trusted for authorization.

2.5 Domain Class Specifications

All fields below are private (−); all listed operations are public (+). The tables expand the diagram rather than replace it. Ancillary operations retain the scope of the prior analysis; detailed algorithms are supplied only for the selected three use cases.

Class and responsibility	Typed attributes	Operation signatures
User (abstract) Identity and role	userID: UUID; name, email, passwordHash: String; role: Role; accountStatus: AccountStatus	login(email: String, password: String): Session logout(sessionID: UUID): void
Faculty Evidence and comments	departmentID: UUID; assignedCourses: Set<UUID>	addComment(aid: UUID, text: String): CommentDTO submitEvidence(aid: UUID, file: Binary): UUID resubmitEvidence(eid: UUID, file: Binary): UUID viewSubmissionStatus(eid: UUID): EvidenceStatus
Auditor, Administrator, HOD, Management Role specializations	Inherited User fields; HOD.departmentID: UUID	Auditor.closeAudit(aid: UUID, version: int): ClosureDTO Administrator.assignRole(uid: UUID, role: Role): void HOD.assign(actionID: UUID, facultyID: UUID): void Management.viewDashboard(): DashboardDTO
Audit Aggregate root for closure	auditID: UUID; title: String; status: AuditStatus; version: int; closedAt: Instant?; closedBy: UUID?; criteria: List<AuditCriterion>	isSatisfied(): boolean updateStatus(status: AuditStatus): void
AuditCriterion Evaluation of an audit requirement	criterionID, auditID: UUID; name, category, requirement: String; priority: Priority; outcome: EvaluationOutcome?	evaluate(result: EvaluationOutcome): void
Evidence Versioned supporting record	evidenceID, auditID, criterionID, facultyID: UUID; title, fileType, fileRef: String; status: EvidenceStatus; version: int	upload(file: Binary): UUID download(): Binary review(result: EvidenceStatus): void resubmit(file: Binary): UUID
Observation Finding and verification state	observationID, auditID, evidenceID: UUID; description: String; severity: Severity; status: ObservationStatus	create(description: String): UUID updateStatus(status: ObservationStatus): void reopen(reason: String): void
CorrectiveAction Remediation and deadlines	actionID, observationID, assignedTo, assignedBy: UUID; description: String; dueDate: Date; priority: Priority; status: ActionStatus	assign(facultyID: UUID): void submit(evidenceID: UUID): void approve(): void checkOverdue(today: Date): boolean

2.6 Comment, Notification and Report Design

Class	Typed attributes	Operations and responsibility
Comment	commentID, auditID, authorID: UUID; observationID, attachmentID: UUID?; text: String; timestamp: Instant; requestID: UUID	store(): UUID; display(): CommentDTO. Preserves author and comment history; optional observation link realizes FR-027.
Notification	notificationID, auditID, recipientID, eventID: UUID; message: String; timestamp: Instant; isRead: boolean; type: Channel; deliveryStatus: DeliveryStatus	send(): DeliveryResult; markAsRead(): void. One recipient-specific inbox record per event; per-channel delivery tracked separately.
AuditReport	reportID, auditID: UUID; summary: String; closureDate: Instant?; snapshot: ReportData	generate(audit: Audit): AuditReport; export(format: ReportFormat): Binary. Captures criteria, evidence, findings, actions and closure state.
CommentVote	commentID, userID: UUID; value: int constrained to −1 or +1	setValue(value: int): void. Composite key prevents multiple simultaneous votes by the same user; realizes A3/A5 alternative.
ActivityLog	logID, auditID, userID: UUID; action: String; previousStatus, newStatus: String; timestamp: Instant	append(entry: ActivityEntry): UUID. Append-only persistence detail derived from FR-047 and NFR-020.

2.6.1 Data structures and value types

Type	Fields or values and use
Session	sessionID: UUID; userID: UUID; expiresAt: Instant. Server resolves the active account.
CommentRequest / CommentDTO	Request: requestID, text or attachmentID, optional observationID. Result: commentID, author display name, text, timestamp, persistenceStatus (STORED/PENDING).
NotifyRequest / DeliveryResult	Request: auditID, recipientIDs: Set<UUID>, message, channels: Set<Channel>, requestID. Result: eventID, recipientCount, deliveryStatus.
ClosureDTO / Blocker	Result: auditID, status, closedAt, closedBy, reportID, version, notificationStatus. Blocker: objectID, reason.
Enumerations	Role: FACULTY, AUDITOR, ADMINISTRATOR, HOD, MANAGEMENT. AccountStatus: ACTIVE/INACTIVE. EvaluationOutcome: COMPLIANT, PARTIALLY_COMPLIANT, NON_COMPLIANT, NOT_APPLICABLE.
Lifecycle values	AuditStatus: DRAFT, EVIDENCE_COLLECTION, CORRECTIVE_ACTION, CLOSURE_REVIEW, CLOSED. EvidenceStatus: PENDING_REVIEW, ACCEPTED, REJECTED, REVISION_REQUIRED. ObservationStatus: OPEN/CLOSED. ActionStatus: ASSIGNED, SUBMITTED, APPROVED, REJECTED. These names formalize the existing workflow.
Delivery and collections	Channel: BELL, PUSH, EMAIL, POPUP. DeliveryStatus: PENDING, SENT, FAILED. Lists retain time/version order; recipient sets remove duplicates. Priority and Severity use LOW/MEDIUM/HIGH.

2.7 Relationships and Persistent Data

Relationship	Multiplicity and implementation
Generalization	Faculty, Auditor, Administrator, HOD and Management inherit User. A users table stores common fields with role as discriminator.
Composition	Audit 1 contains 0..* AuditCriterion and 0..* Comment. Each criterion/comment belongs to exactly one audit. Historical records are retained; composition does not authorize deletion.
Association	Auditor 1..* ↔ Audit 1..* for assigned active audits, through audit_assignments. Draft/unassigned records may temporarily have zero assignments; closure requires an authorized assigned auditor or administrator.
Association	Faculty 1 → 0..* Evidence; Audit 1 → 0..* Evidence/Notification/Report. Each notification has one recipient User; each comment has one author User.
Aggregation	Evidence 1 ◇— 0..* Observation. The finding remains a historical record if a newer evidence version is submitted; any relinking retains history.
Dependency	AuditReport depends on Audit to generate the report. CorrectiveAction depends on Observation; persisted observationID gives each action exactly one finding. HOD assigns actions to faculty.

2.7.1 Relational storage design

Table or store	Keys, constraints and persistent content
users; audit_assignments	users PK user_id; UNIQUE(email). Assignments PK(audit_id, auditor_id); both are foreign keys.
audits; audit_criteria	PK audit_id / criterion_id. Criterion FK audit_id. Audit stores status, version, closed_at, closed_by; version supports concurrency checks.
evidence; observations; corrective_actions	Evidence retains numbered versions and protected file references. Observation FKs audit_id/evidence_id; action FKs observation_id/assigned_to/assigned_by. No overwrite of prior evidence.
comments; comment_votes	comments PK comment_id, FKs audit_id/author_id and optional observation_id; UNIQUE(author_id, request_id). votes PK(comment_id, user_id), CHECK value IN (−1,1).
notifications; delivery_jobs	notifications UNIQUE(event_id, recipient_id). Jobs UNIQUE(notification_id, channel). Store attempts, status and next_attempt_at; inbox reads notifications, not external delivery responses.
audit_reports; activity_logs	Report FK audit_id; closure report is tied to the closing version. Activity log stores actor, time, before/after state. Closure report and log are written with audit status.
Protected files and pending comments	Evidence/comment attachments stored by generated ID outside public web paths. Durable filesystem journal stores pending CommentCommand plus user, audit and request IDs; replay rechecks access and avoids duplicate inserts.

Indexes: comments(audit_id, timestamp); notifications(recipient_id, is_read, timestamp); observations(audit_id, status); corrective_actions(observation_id, status, due_date). All referenced objects must belong to the same audit where applicable. FK deletion is restricted for historical records.

3. Architectural Design

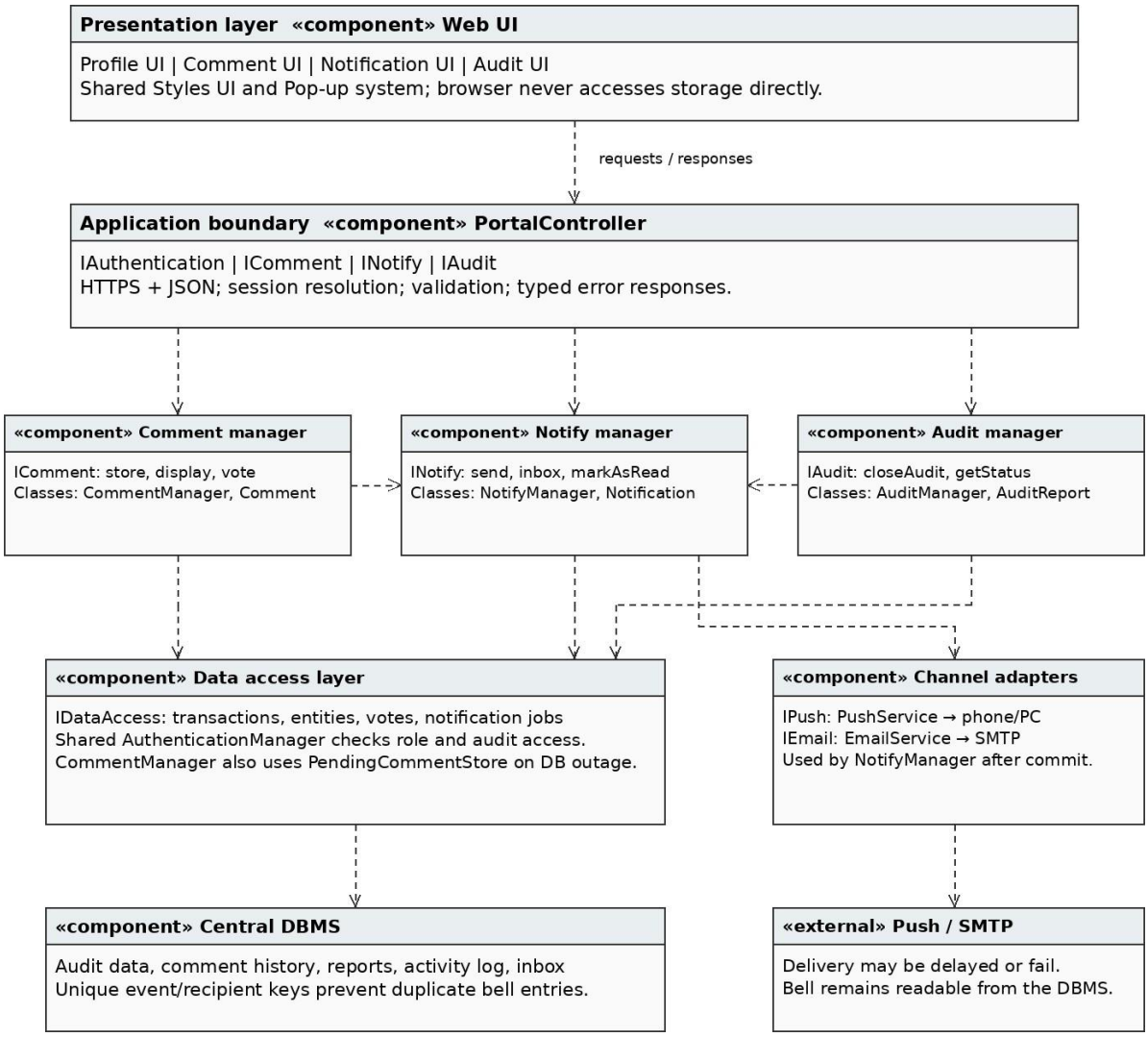


Figure 2: UML component representation of the layered client–server architecture

3.1 Architectural style and justification

A layered client–server application with a modular server is selected. Browser screens call the controller; the controller resolves authentication and invokes managers; managers use DataAccess; DataAccess accesses the centralized DBMS. This follows the A5 layers and NFR-014. One server codebase is appropriate for the student team and allows a single database transaction for closure, report, history and notification persistence.

UI, business logic and storage can be tested independently. Push and email adapters isolate external delivery failures. The DBMS remains authoritative; the filesystem journal is used only for the existing comment-outage alternative. Phone/PC alerts use the browser notification channel, not an out-of-scope mobile application.

3.2 Architecture Responsibilities and Behaviour

Layer or subsystem	Responsibility and communication
Presentation	Profile UI signs in; Comment UI lists/stores comments and votes; Notification UI sends and reads alerts; Audit UI reviews and closes audits. Styles UI and Pop-up system provide consistent feedback.
Controller and authentication	Validate HTTP inputs, resolve server session, enforce role and audit-level access, map domain results to DTOs and errors. No user screen calls DBMS directly.
Comment manager	Validate comment content, optional observation linkage and attachment fallback; save comment/history; enqueue notification jobs in the same transaction. Filesystem fallback returns PENDING rather than a false success.
Notify manager	Resolve eligible recipients, deduplicate event/recipient records, maintain inbox/read status, deliver push or email. Automatic triggers originate from comments, resubmissions and overdue actions; manual notify is retained.
Audit manager	Load latest audit, verify closure readiness, authorize the actor, atomically close and write report/history/notification jobs. Concurrent updates cannot overwrite a completed closure.
Data access and external services	Parameterized DB access and transaction boundaries. Push/SMTP adapters accept recipient/message data from NotifyManager; they no longer query the DB independently.

3.3 Use-case realization and failure isolation

UC-003: Comment UI → IComment → CommentManager → DataAccess → DBMS. Commit stores the comment and notification jobs before showing “Commented!”. DB failure invokes PendingCommentStore; the user sees a pending message. UC-005 is then dispatched through NotifyManager.

UC-005: Notification UI or a validated system event → INotify → NotifyManager → persisted inbox + delivery jobs. A stable eventID gives one bell entry per recipient. External calls occur after commit. Push failure attempts email; if both fail, retain a failed/pending delivery record for retry while preserving the bell entry.

UC-009: Audit UI → IAudit → AuditManager → latest criteria, evidence, observations and actions → transaction containing Closed status, closure date, report, log and notifications. A stale version returns a conflict with no update. Failure before commit leaves the audit open; failure after commit in push/email does not reopen it.

3.4 Refinements to Assignment 5

Success pop-ups are emitted after a confirmed commit, not before persistence. The exclusive manual/automatic notification branches are a decision/merge, not a concurrency fork/join; channel delivery can then proceed independently. AuditManager explicitly depends on DataAccess. Channel adapters receive data from NotifyManager rather than fetching it themselves. These corrections preserve use-case behaviour and clarify implementation ordering.

3.5 Design consistency

Comment maps to FR-027, BR-004/006 and A5 Comment collaboration/state models. Notify maps to FR-28, BR-005/007/011/012 and A5 Notify models. Close audits maps to FR-012/022/033/034, BR-002/003 and A5 closure collaboration. FR-042/046/047 justify report and history persistence. Assignment 7 gives the complete interface and operation traceability matrix.

4. Contributions and Declaration

Individual Contribution

Proposed allocation for team review. Each member should confirm or amend this table to reflect the work they actually complete before submission.

Name	Roll number	Assigned contribution
Ayush Gupta	24ucs261	Refine domain class attributes, operations and UML class view.
Aayana Jain	24ucs262	Review relationships, multiplicities, persistent keys and consistency.
Daksh Jain	24ucs263	Define architecture, manager interfaces and failure behaviour.
Dhruv Shanker Dandotiya	24ucs264	Review use-case realization, traceability and document integration.

AI Usage Policy — Student Declaration

AI tools may be used as supporting tools. The team is responsible for the correctness, consistency and understanding of all submitted design artifacts. The following declaration must be confirmed by the team after review:

• We have disclosed the use of AI tools, if any.

AI-generated suggestions have been reviewed, verified and adapted by the team.

• We understand and can explain the submitted design class diagram, architecture, interfaces and component specifications.

• We have verified consistency with our project requirements, use cases and analysis models.

• The final submission represents our team’s understanding and design decisions.

AI Tool(s) Used: ChatGPT

Purpose of AI Use: Assistance with design refinement, diagrams, wireframes, interface specifications
Team ID: T094
Team Representative: Dhruv Shanker Dandotiya

The LNM Institute of Information Technology, Jaipur
Software Development Laboratory
Course Code: CS314

Lab Assignment 7: Solution

Software Design Modelling
T094 – Academic Audit Portal

Item	Details
Team members	Ayush Gupta, Aayana Jain, Daksh Jain, Dhruv Shanker Dandotiya
sRoll numbers	24ucs261, 24ucs262, 24ucs263, 24ucs264
Selected use cases	UC-003 Comment; UC-005 Notify; UC-009 Close audits

1. Objective

To specify the user and software interfaces of the Academic Audit Portal, explain how each selected use case is implemented internally, and trace the design back to the established requirements and models. The component and operation names are the same as those in Assignment 6.

2. Interface Design

2.1 Screens and navigation

Screen	Purpose and navigation
W01 Sign in	Authenticate → role dashboard → select an authorized audit. Expired sessions return here.
W02 Audit details and comments	Read audit summary; add comment or file; view author profile; vote. Navigate to W03, W04 or W05.
W03 Compose notification	Select authorized recipients, message and channels; Send/Cancel → selected audit. Automatic triggers do not require this screen.
W04 Notification inbox	Bell opens inbox; mark read; follow a link to W02 or W06.
W05 Closure review	Show readiness checks and blockers; confirm close → W06. Cancel returns to audit; Notify users opens W03.
W06 Closed audit	Show status, closure date, actor and report; return to dashboard or W02 to comment.

Wireframes use illustrative audit identifiers and values. Every protected screen enforces server-side authorization. Existing audit criteria are read-only for faculty (BR-010). A hidden button is only a usability aid, not an authorization mechanism.

2.2 Wireframes for Authentication and Comments

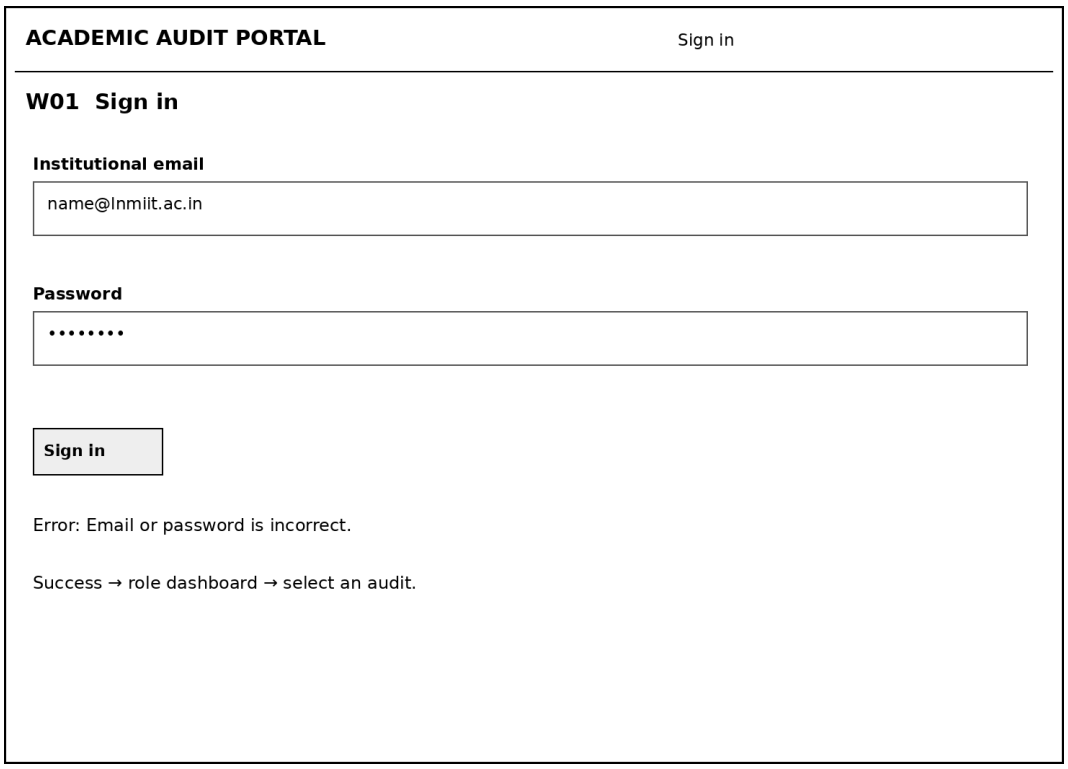


Figure 1: W01 Sign in

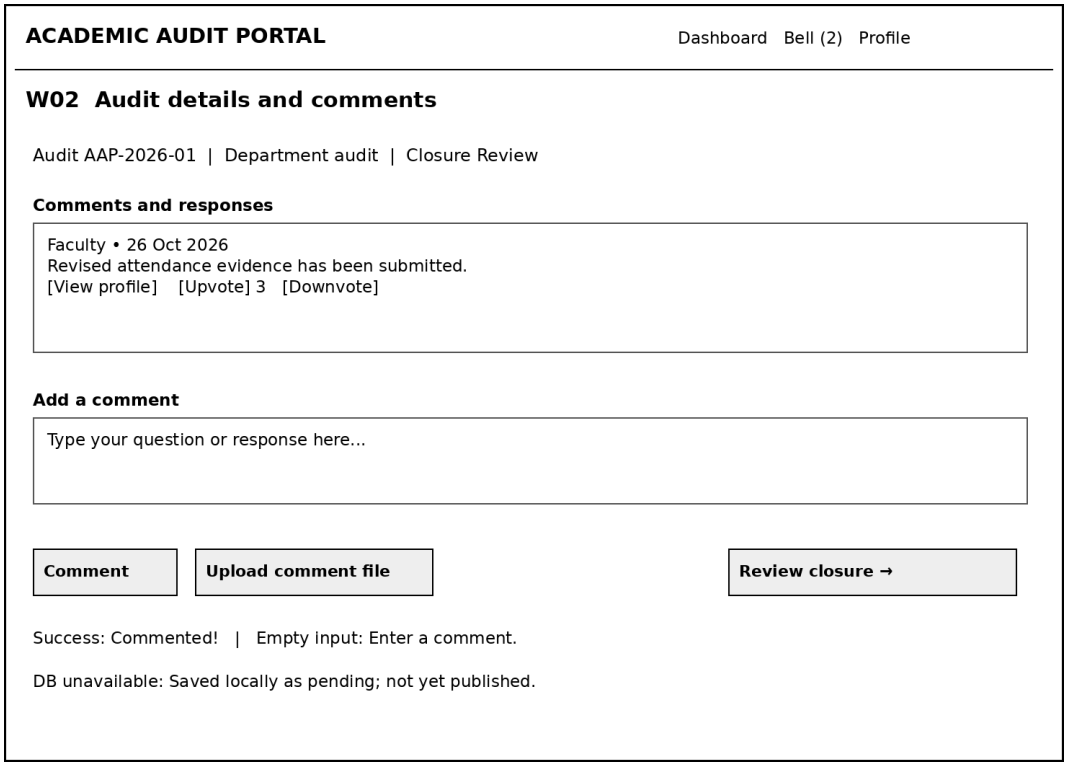


Figure 2: W02 Audit details and comments — UC-003

Blank comments are rejected. The file option supports the existing non-working-comment-box alternative; it is not a general evidence-upload workflow. After a DB outage, the screen distinguishes durable pending storage from publication.

2.3 Wireframes for Notifications

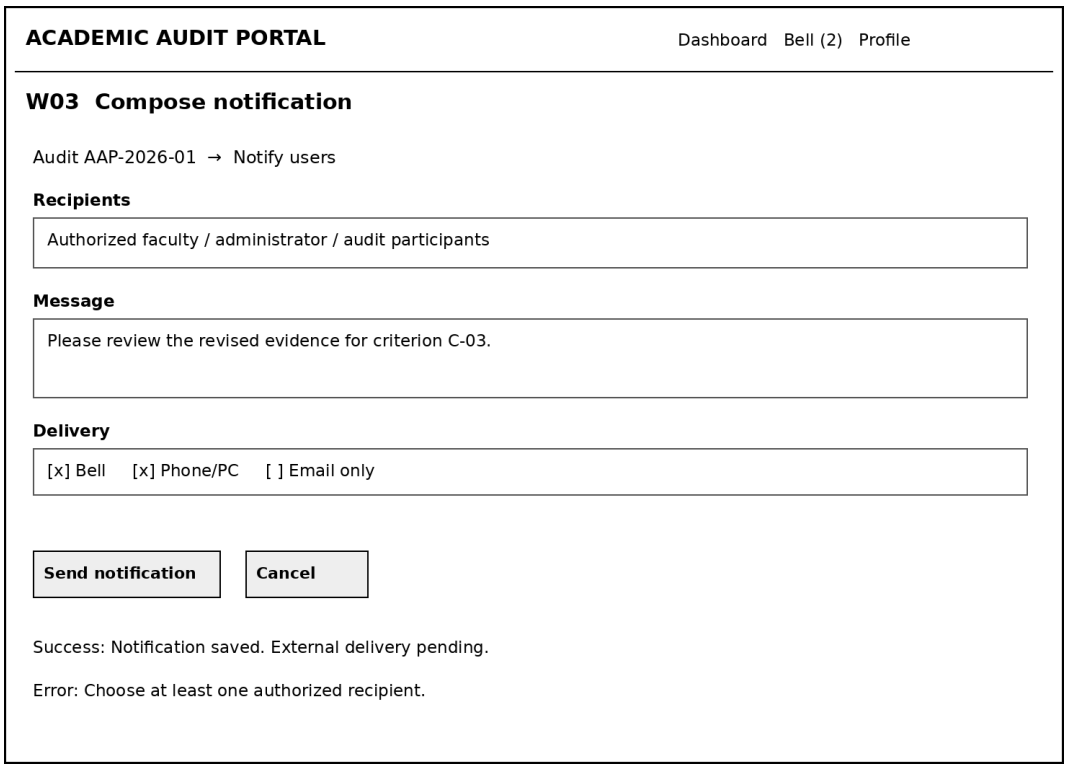


Figure 3: W03 Manual notification — UC-005

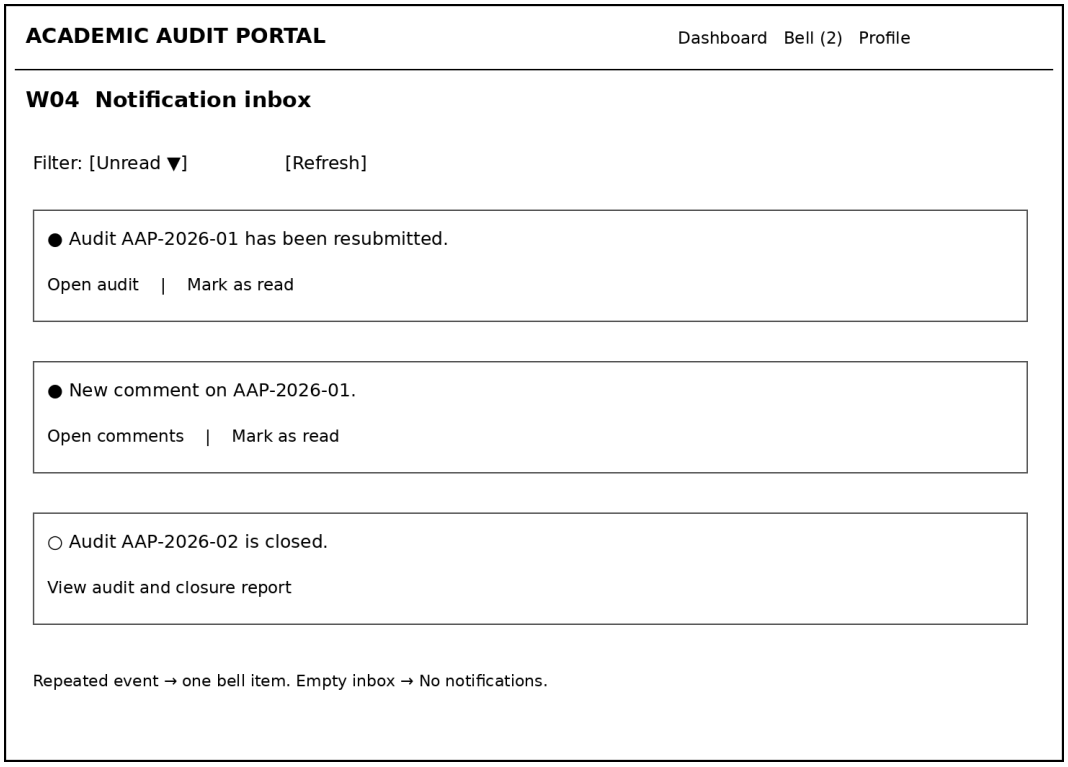


Figure 4: W04 Bell inbox — UC-005

Notification content links back to the audit. Duplicate events share one inbox entry. Marking a notification as read changes only that recipient's record. Browser push permission may be denied; bell delivery remains available and email is the existing fallback.

2.4 Wireframes for Audit Closure

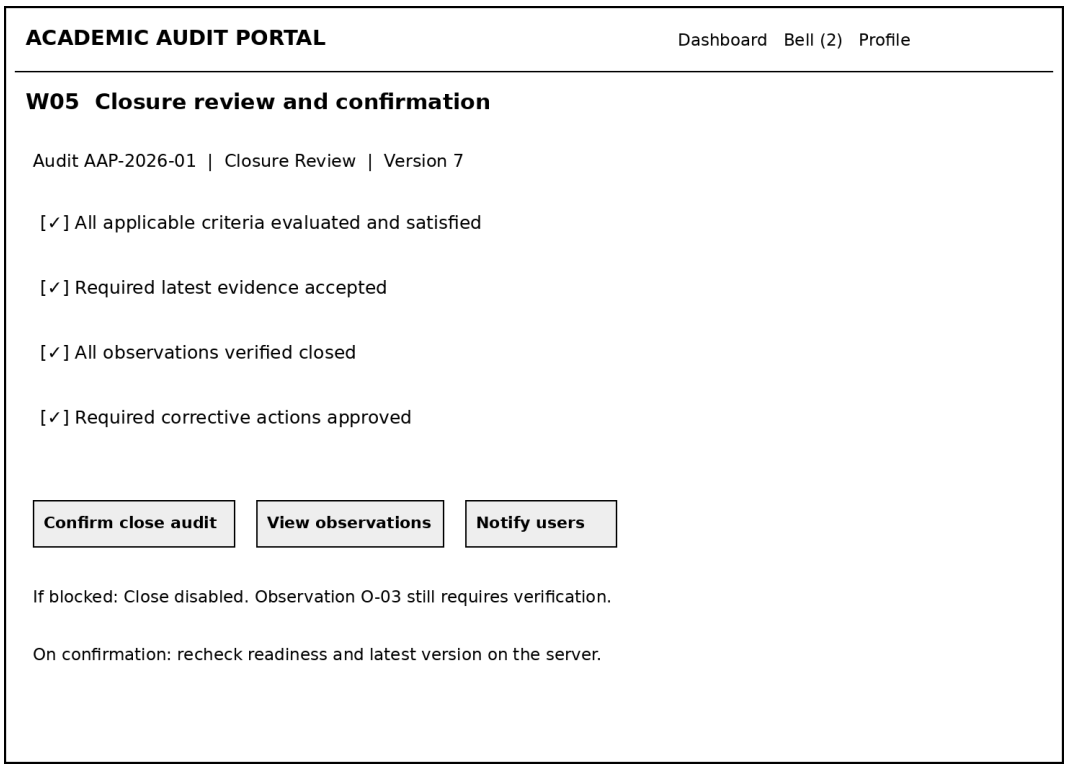


Figure 5: W05 Readiness and closure confirmation — UC-009

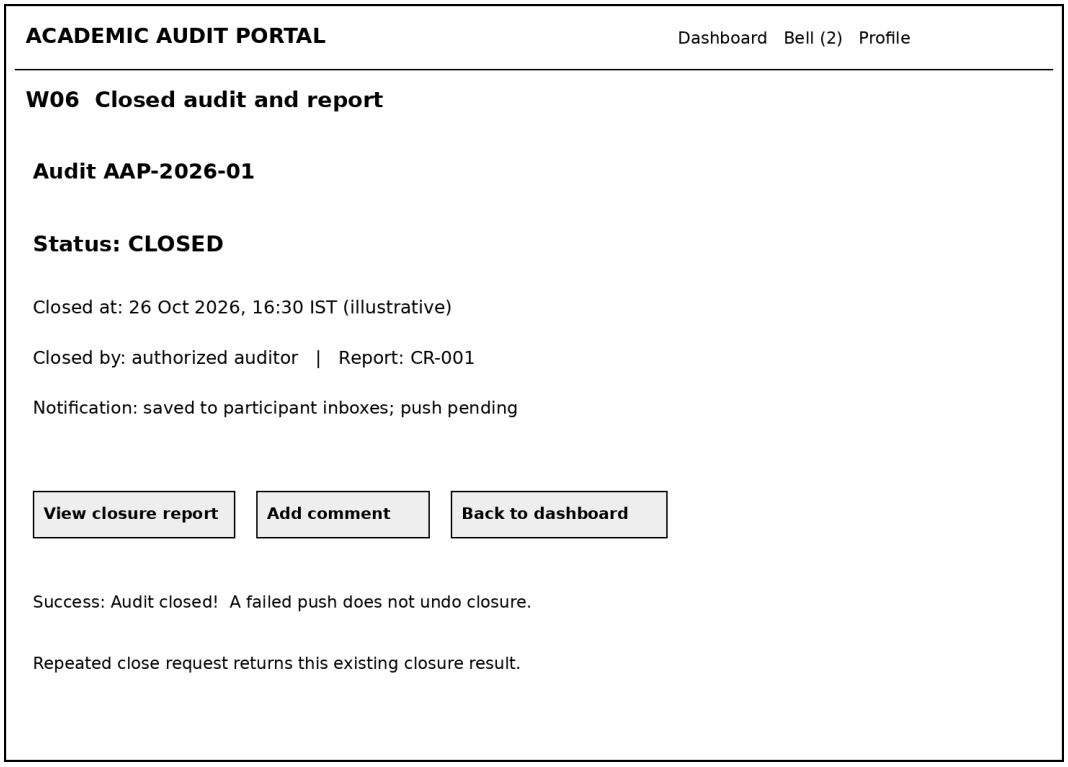


Figure 6: W06 Successful closure and report — UC-009

Readiness shown on screen is advisory. The server rechecks it against the latest audit when Confirm close audit is pressed. Faculty may view status and comment but cannot close an audit. The confirmation action is disabled while a request is in progress.

2.5 Validation and Error Messages

Scenario	Validation or processing rule	User feedback
Login fails or session expires	Verify active account and session on every request. Do not disclose whether an email exists.	“Email or password is incorrect.” / “Your session expired. Please sign in again.”
Blank or oversized comment	Trim text. Proposed implementation limit: 2,000 characters; accept text or a valid attachment. Escape output as text.	“Enter a comment or upload a comment file.” / “Comment must be 2,000 characters or fewer.”
File fallback	Proposed limits: PDF/TXT, up to 5 MB; inspect actual file type, use generated storage IDs, require authorized retrieval.	“Choose a PDF or TXT file up to 5 MB.”
Comment control unavailable	UI shows a retry or file option if the form fails. Never display success merely because the button was clicked.	“Comment could not be sent. Retry or use a comment file.”
DB unavailable	Durably journal only after access was validated; return 202 PENDING. If identity/access cannot be checked or journal fails, reject safely.	“Saved locally as pending; not yet published.” / “Unable to save. Keep a copy and retry.”
Manual notification invalid	Require nonblank message and at least one eligible recipient; deduplicate recipient IDs. Proposed message limit: 2,000 characters.	“Enter a message.” / “Choose at least one authorized recipient.”
External notification failure	Preserve inbox; attempt email on push failure. A failed email remains recorded for retry.	“Saved in the bell. External delivery delayed.”
Duplicate trigger	Use a stable server eventID, not message text, as the identity.	Existing bell entry remains; no second entry is added.
Closure blocked	Reject if latest applicable criteria/evidence or verified findings/actions do not meet closure rules. Return blocker IDs.	“Cannot close: observation O-03 requires verification.”
Version changed or already closed	Lock audit and compare expectedVersion; repeated closure returns existing result.	“Audit changed. Refresh and review again.” / “Audit already closed.”
Permission or storage error	403 for disallowed operation; 404 for unavailable resource; 503 for unavailable storage; no SQL details in UI.	“You are not allowed to perform this action.” / “Service unavailable. Please retry.”

2.5.1 Common interface rules

HTTPS carries JSON requests and responses. Sessions use secure, HttpOnly cookies; state-changing requests require CSRF protection. Identity comes from the session, never the request body. Input limits above are implementation choices for validation, not new SRS requirements. All dates are stored in UTC and displayed in the user’s local time.

Errors use {code, message, fieldErrors?, blockers?, requestID}. requestID correlates retries; it is distinct from the server-generated business eventID. UI drafts are retained after a failed request so the user can retry without retyping.

2.6 Software Interface Specification

Interface and provider	Operation signature	Inputs, outputs and errors
IAuthentication AuthenticationManager	resolve(session: Session): User authorize(user: User, auditID: UUID, operation: String): void	Active server session; audit assignment/role. Returns authorized User or 401/403. Profile response exposes only authorized name/role.
IComment CommentManager	store(user: User, aid: UUID, req: CommentRequest): CommentDTO display(user: User, aid: UUID): List<CommentDTO> vote(user: User, cid: UUID, value: int): VoteDTO	Text/attachment, observationID?, requestID. Result STORED/PENDING. Reject invalid text/file, foreign observation, invalid vote or inaccessible audit; 400/403/404/503.
INotify NotifyManager	manualSend(user: User, req: NotifyRequest): DeliveryResult send(event: AuditEvent, recipients: Set<UUID>): DeliveryResult markAsRead(user: User, nid: UUID): void	Manual input or trusted event; returns eventID, recipientCount and delivery state. Invalid recipient/message → 400; another user's inbox → 403/404. send is internal, not a public arbitrary-event endpoint.
IAudit AuditManager	closeAudit(user: User, aid: UUID, expectedVersion: int): ClosureDTO getStatus(user: User, aid: UUID): AuditDTO	Assigned auditor/admin, current version. Returns status, closure date, report and notification state. Unauthorized → 403; stale or unsatisfied → 409; DB failure → 503.
IDataAccess DB adapter	loadAudit(id: UUID, forUpdate: bool): Audit saveComment(c: Comment, requestID: UUID): UUID saveClosure(a: Audit, r: AuditReport, l: ActivityLog): void enqueue(n: Notification): void	Transactions cover business records and notification jobs. Typed storage exceptions are mapped by controller; foreign keys and unique keys enforce integrity.
IPendingCommentStore Filesystem adapter	append(command: CommentCommand): UUID readPending(): List<CommentCommand> acknowledge(id: UUID): void	Durable record includes actor/audit/request IDs. Failure returns an error, not a successful pending result. Replay uses normal validation and deduplication.
IPush / IEmail Channel adapters	deliver(recipient: User, message: String, eventID: UUID): DeliveryStatus	Return SENT/FAILED with retryable cause internally. No credentials or provider error details exposed to UI. Call only after DB commit.

Main path: UI → controller → manager interface → domain objects and IDataAccess. CommentManager and AuditManager create notification records through NotifyManager inside the current transaction; the dispatcher delivers them after commit. This avoids sending a success alert for a rolled-back operation.

2.7 HTTP Boundary and Data Contracts

Route	Request and successful response	Relevant failures
POST /api/sessions	{email, password} → 200 session cookie + role; DELETE /api/sessions/current → 204	400 invalid input; 401 failed login
GET /api/audits/{aid}	Authorized session → 200 AuditDTO incl. version and readiness	401/403/404
GET /api/audits/{aid}/comments	Authorized session → 200 ordered CommentDTO list	401/403/404
POST /api/audits/{aid}/comments	{requestID, text?, attachmentID?, observationID?} → 201 STORED; 202 PENDING after durable journal; retry of stored request → 200 existing	400 invalid input; 403 access; 404 audit; 503 no durable save
POST /api/audits/{aid}/comment-files	multipart {file} → 201 {attachmentID}; creates protected temporary attachment, not a published comment	400/413/415; 401/403/503
PUT /api/comments/{cid}/vote	{value: −1 or +1} → 200 VoteDTO with score and userVote	400/403/404
GET /api/users/{uid}/profile	Authorized audit-context view → 200 {name, role}; no password, private contact details	401/403/404
POST /api/notifications	NotifyRequest → 202 DeliveryResult; saved inbox with channel delivery pending	400 recipients/message; 403; 503
GET /api/notifications PATCH /api/notifications/{nid}	GET → 200 own inbox; PATCH {isRead:true} → 200 notification	401/403/404
POST /api/audits/{aid}/close	{expectedVersion} → 200 ClosureDTO; repeat returns same existing closure	403 role/access; 409 STALE_VERSION or CLOSURE_BLOCKED; 503
GET /api/audits/{aid}/closure-report	Authorized session → 200 report snapshot; optional format=pdf exports existing report	401/403/404

2.7.1 Example result contracts

```
CommentDTO = {commentID, auditID, authorName, text, timestamp,
              attachmentID?, persistenceStatus: STORED | PENDING}
DeliveryResult = {eventID, recipientCount, deliveryStatus}
ClosureDTO = {auditID, status: CLOSED, closedAt, closedBy,
              reportID, version, notificationStatus}
Error = {code: CLOSURE_BLOCKED, message: "Audit is not ready",
         blockers: [{objectID: "O-03", reason: "Not verified"}]}
```

IDs in these contracts refer only to records the actor may access. A comment with observationID must reference an observation of the same audit. The server derives author and timestamp; the client cannot forge them. The API returns a specific pending result when the fallback journal has accepted the comment.

3. Component-Level Design

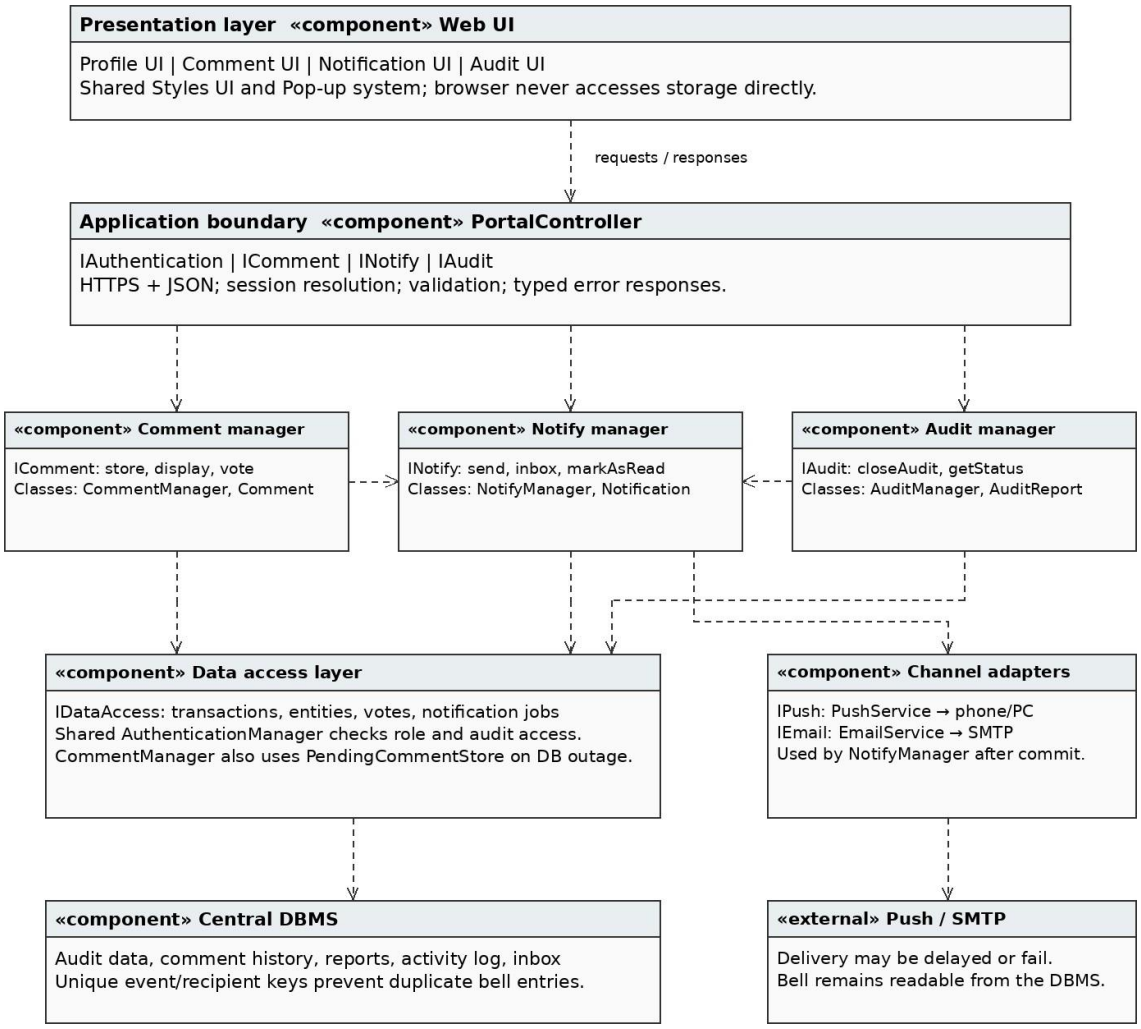


Figure 7: Refined component diagram with named provided interfaces

3.1 Component to class mapping

Component	Classes and important operations
Comment manager	CommentManager.store/display/vote; Comment.store/display; CommentVote.setValue; PendingCommentStore.append/replay support.
Notify manager	NotifyManager.manualSend/send/dispatchPending; Notification.send/markAsRead; PushService.deliver; EmailService.deliver.
Audit manager	AuditManager.closeAudit/validateClosure; Audit.isSatisfied/updateStatus; AuditCriterion.evaluate; AuditReport.generate; ActivityLog.append.
Shared infrastructure	PortalController; AuthenticationManager.resolve/authorize; DataAccess transaction methods. Operation names agree with Assignment 6.

The refined diagram preserves the A5 components. Interface labels identify the services each manager provides; arrows represent client-to-supplier dependencies. The shared controller and explicit fallback store refine implementation structure without adding a user-visible feature.

3.2 CS-01 Comment Processing — UC-003

Specification	Details
Entry and responsibility	IComment.store(user, auditID, request) → CommentDTO. CommentManager coordinates validation, history, persistence and notification.
Preconditions	Active authenticated user; authorized access to a published audit; valid text or uploaded comment file. Observation link, if present, belongs to the audit.
Postconditions	Normal: comment and notification jobs committed once, visible in audit history. DB-outage alternative: durable pending command stored, explicitly not yet visible as a published comment.
Dependencies	AuthenticationManager, Audit/Comment, DataAccess, NotifyManager, PendingCommentStore; optional CommentVote for voting.

3.2.1 Processing logic

```
1. Resolve session; authorize COMMENT on auditID.
2. Trim text; validate text/file and optional observationID.
3. If (authorID, requestID) already exists, return existing comment.
4. Begin transaction; load audit and validate current access/state.
5. Create Comment(authorID from session, timestamp from server).
6. Save Comment; eventID := stable ID derived from saved commentID.
7. NotifyManager.send(COMMENT_ADDED event, authorized audit users)
   inserts deduplicated inbox records and delivery jobs in this tx.
8. Commit; return STORED. UI adds comment and shows "Commented!".
9. On a confirmed DB failure, rollback; append the validated command
   to PendingCommentStore with the same requestID. Return PENDING.
10. If durable append fails, return 503; do not claim it was saved.
11. If commit outcome is unknown, retain the command for reconciliation;
    replay checks the unique requestID before inserting anything.
12. Dispatcher sends channel alerts only for committed records.
```

3.2.2 Alternatives and recovery

The filesystem fallback is used only when identity and audit access have already been validated. Otherwise reject rather than accept an unauthorized pending command. The journal is private, durable and not executable. Replay rechecks authorization, audit linkage and validation, stores the comment and notification jobs transactionally, then acknowledges the journal record after commit. If replay crashes, the same requestID prevents a duplicate comment.

If the textbox fails, W02 accepts a validated file reference. If the button fails, show retry/file options without claiming success. Comment.display returns escaped text and authorized attachment links. Profile views expose the commentor's permitted name and role. vote accepts −1/+1 and upserts one row per (commentID, userID); score is the sum of saved votes.

3.2.3 Verification scenarios

Valid comment → one stored comment and one event. Blank comment → no write. Same request retried → existing comment. DB down after authorization → pending journal entry and no success pop-up. Replay twice → one comment. Foreign observationID or unauthorized attachment → rejection. These are proposed checks, not claims of executed software tests.

3.3 CS-02 Notification Processing — UC-005

Specification	Details
Entry and responsibility	INotify.manualSend(user, request) or internal send(event, recipients) → DeliveryResult. NotifyManager persists, deduplicates and dispatches alerts.
Preconditions	Manual sender is authenticated and authorized for the audit/recipient set. Automatic trigger is a validated server-side comment, resubmission, overdue-action or closure event.
Postconditions	Exactly one bell record per event and recipient; external delivery result tracked independently. No notification references a rolled-back business operation.
Dependencies	Notification, DataAccess, AuthenticationManager, PushService, EmailService. UI Pop-up system renders a committed result.

3.3.1 Processing logic

```
1. For manual send: authenticate; validate message and recipients.
   Bind requestID to sender/audit; create or reuse its eventID.
2. For automatic send: use the source record and event type/version
   to obtain a stable eventID (never trust a client-supplied event).
3. Resolve eligible recipients; remove duplicate recipient IDs.
4. Within caller transaction, or a new tx for manual send:
   for each recipient:
     insert Notification unless (eventID, recipientID) exists;
     enqueue missing selected channels; do not reset isRead.
5. Commit only if this operation owns the transaction.
6. Return saved inbox/delivery state; dispatcher runs after commit.
7. Dispatcher claims a pending channel job with a lease.
8. Deliver push; if it fails, enqueue/attempt email fallback once.
   If email-only selected, call EmailService directly.
9. Persist per-channel SENT/FAILED and retry metadata.
10. On temporary failure, retry with bounded backoff (design choice:
    up to 3 attempts per channel); retain final failure for inspection.
11. markAsRead checks recipient ownership before updating isRead.
```

3.3.2 Decisions and exceptional cases

Manual and automatic triggers are alternatives; they merge into one persistence path. After persistence, bell, pop-up and external delivery do not need to succeed simultaneously. A retry must reuse the source eventID. Two different comments with identical text are two legitimate events. A delayed notification uses its stored due time, preserving the existing NFR-002 behaviour.

If the notification database cannot commit, manual send returns 503 and does not report success. In comment/closure transactions, failure rolls back the related business transaction; the comment use case may then enter its defined filesystem fallback. External push/email failures occur after commit and never undo the related business action. External delivery is at least once; provider retries can duplicate an email, while the database unique key guarantees one bell record.

3.3.3 Verification scenarios

Repeated source event → one bell item; distinct events → separate items. Push blocked → email fallback. Push and email fail → bell retained, delivery marked failed. Another user marks this recipient's alert read → denied. Crash after external send → retry may redeliver externally but cannot create a second bell item.

3.4 CS-03 Audit Closure — UC-009

Specification	Details
Entry and responsibility	IAudit.closeAudit(user, auditID, expectedVersion) → ClosureDTO. AuditManager validates and commits the closure.
Preconditions	Assigned auditor or authorized administrator; current audit version; readiness conditions satisfied. Faculty authentication alone does not authorize closure.
Postconditions	Audit is Closed with actor/date, report snapshot and activity log; notification jobs exist. On failed validation/transaction, no partial closure remains.
Dependencies	AuthenticationManager, Audit, AuditCriterion, Evidence, Observation, CorrectiveAction, AuditReport, ActivityLog, DataAccess, NotifyManager.

3.4.1 Readiness rule

“Audit is satisfied” is refined as: audit is in Closure Review; every applicable criterion is evaluated as Compliant (Not Applicable is excluded); required latest evidence is accepted; all associated observations are verified Closed; all required corrective actions are Approved. Partial/non-compliance and pending verification block closure. This operationalizes the existing UC-009 precondition using FR-019/022/032/034; it does not authorize an administrator to bypass auditor verification.

3.4.2 Processing logic

```
1. Resolve session; authorize CLOSE for assigned auditor/admin.
2. Begin transaction; lock audit row (loadAudit(aid, forUpdate=true)).
3. If status = CLOSED, return existing closure after ending tx.
4. If version != expectedVersion, rollback; return STALE_VERSION.
5. Load latest criteria, evidence, observations and actions.
6. blockers := validateClosure(audit).
   If blockers not empty, rollback; return CLOSURE_BLOCKED(blockers).
7. Set status = CLOSED, closedAt = nowUTC, closedBy = user.userID;
   increment version; generate immutable closure report snapshot.
8. Append activity entry with actor and old/new status.
9. NotifyManager.send(AUDIT_CLOSED event, authorized participants)
   writes inbox records and delivery jobs within this transaction.
10. Save audit, report and log; commit all changes atomically.
11. Return ClosureDTO; W06 shows "Audit closed!", date and report.
12. On storage exception, rollback; return 503 without success.
    On uncertain commit outcome, reread closure before retrying.
```

3.4.3 Concurrency and alternatives

All writes to closure-relevant child records acquire the same audit lock and increment its version; otherwise locking only the audit would not prevent a concurrent evidence/observation change. A competing close sees the committed closure and returns it. Close control failure leaves status unchanged; the user can open W03 to notify participants, but that message must not claim the audit was closed.

Comment failure after successful closure does not undo closure. The closed audit remains viewable with its report and comment history. Proposed checks: faculty closure denied; open observation blocks closure; stale version rejected; DB failure rolls back report/status/log together; concurrent double-close produces one closure; push outage leaves the committed audit Closed.

4. Design Consistency and Traceability

The complete chain is: SRS → selected use case → A3 scenario/activity → A4 classes/relationships → A5 collaboration/state/component model → A6 design classes/architecture → A7 interface → component logic. The table preserves baseline identifiers and names.

Use case and requirements	Analysis and A5 evidence	A6 design → A7 interface → logic
UC-003 Comment FR-027; BR-004/006; US-17	A3 Comment scenario; A4 User/Faculty, Audit, Comment and composition; A5 Comment collaboration and query-state lifecycle.	CommentManager, Comment, CommentVote, PendingCommentStore → W02, IComment, comment routes → CS-01.
UC-005 Notify FR-28; BR-005/007/011/012; US-05	A3 Notify scenario including duplicate and email fallback; A4 Notification/Audit/User; A5 bell, pop-up, manual/automatic triggers and channels.	NotifyManager, Notification, PushService, EmailService → W03/W04, INotify → CS-02; source event identity prevents duplicate bell entries.
UC-009 Close audits FR-012/022/033/034; BR-002/003	A3 closure sequence; A4 auditor1/audit1/report1/notif1/faculty1 object scenario; A5 Close button, status, closure date and notifications.	AuditManager, Audit, AuditCriterion, Observation, CorrectiveAction, AuditReport → W05/W06, IAudit → CS-03.
Shared access and history FR-001/005/042/046/047; NFR-004/006/011/012/014/020	A4 abstract User and subclasses; A5 Profile UI, Authentication manager, Data access layer and DBMS.	PortalController, AuthenticationManager, DataAccess, ActivityLog → W01 plus all guarded routes → authentication, transactions, append-only history.

4.1 Consistency decisions

UC-003 refers to Comment despite the duplicate ID in the early catalogue. FR-28 remains spelled as in the baseline. Audit-level comments from the detailed use case coexist with optional observation-linked comments under FR-027. Selected use cases and user-visible scope are unchanged.

The A5 early success pop-up is refined to occur after commit. The Notify state model's exclusive trigger selection is implemented as decision/merge, while channel dispatch can be concurrent. Pending-comment persistence implements the A2 database-crash alternative that was abbreviated as DB Error in A5. An administrative role does not bypass the closure verification rule.

Class diagram signatures, manager responsibilities, HTTP payloads and pseudocode use the same types and IDs. Storage uniqueness enforces comment retry safety and notification deduplication. Closed status, report and closure date derive from one committed transaction. Bell data persists independently of external channel delivery.

4.2 Source documents and submission organization

Sources: T-094_LAB_Assignment4_5.docx (T094 baseline including A1–A5); Assignment-6-7(1).pdf (task instructions); T018_Lab_Assignment_6_Solution(1).docx and T018_Lab_Assignment_7_Solution(1).docx (structure/reference only). T018 project requirements are not imported into T094.

For the later repository exercise, organize design content under docs/design/assignment-06/ using Data-Class-Design.md, Architectural-Design.md, Interface-Design.md, Component-Level-Design.md, Design-Traceability.md and diagrams/. The supplied PDFs are the submission documents; no repository commit or Classroom submission is claimed.

5. Contributions and Declaration

Individual Contribution

Proposed allocation for team review. Each member should confirm or amend this table to reflect the work they actually complete before submission.

Name	Roll number	Assigned contribution
Ayush Gupta	24ucs261	Review W01/W02, comment interface and CS-01 processing logic.
Aayana Jain	24ucs262	Review W03/W04, notification interface and CS-02 processing logic.
Daksh Jain	24ucs263	Review W05/W06, closure interface and CS-03 processing logic.
Dhruv Shanker Dandotiya	24ucs264	Review refined component diagram, traceability and final integration.

AI Usage Policy — Student Declaration

AI tools may be used as supporting tools. The team is responsible for the correctness, consistency and understanding of all submitted design artifacts. The following declaration must be confirmed by the team after review:

• We have disclosed the use of AI tools, if any.

• AI-generated suggestions have been reviewed, verified and adapted by the team.

• We understand and can explain the submitted design class diagram, architecture, interfaces and component specifications.

• We have verified consistency with our project requirements, use cases and analysis models.

• The final submission represents our team’s understanding and design decisions.

AI Tool(s) Used: ChatGPT
Purpose of AI Use: Assistance with design refinement, diagrams, wireframes, interface specifications, processing logic
Team ID: T094
Team Representative: Dhruv Shanker Dandotiya